

Data-Level Parallelism Architecture

Data-Level Parallelism

Data-Level Parallelism

exploit parallelism from simultaneous operations across large sets of data rather than from multiple threads of control

Data-Level Parallelism

potentially more energy-efficient:
a single instruction can launch
many data operations

Data-Level Parallelism

easier on the programmer than
classic parallel MIMD programming

Data-Level Parallelism

example applications:

matrix-oriented scientific computing

media-oriented image & sound processing

machine learning

Data-Level Parallelism

example architectures:

Vector Architecture

Multimedia SIMD

GPU

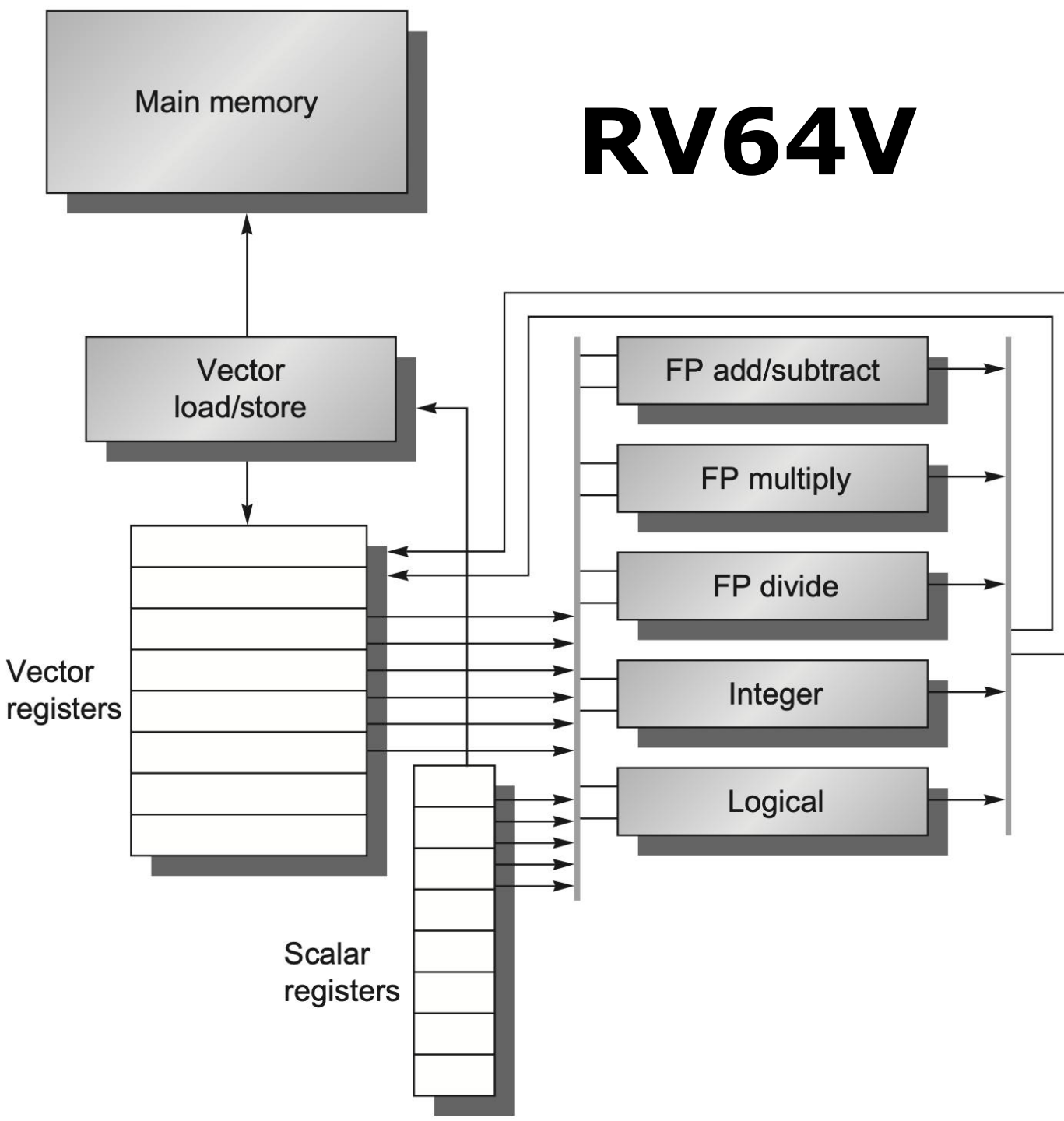
Vector Architecture

- Grab sets of data elements scattered about memory
- Place them in large sequential reg files
- Operate on data in those reg files
- Disperse the results back into memory

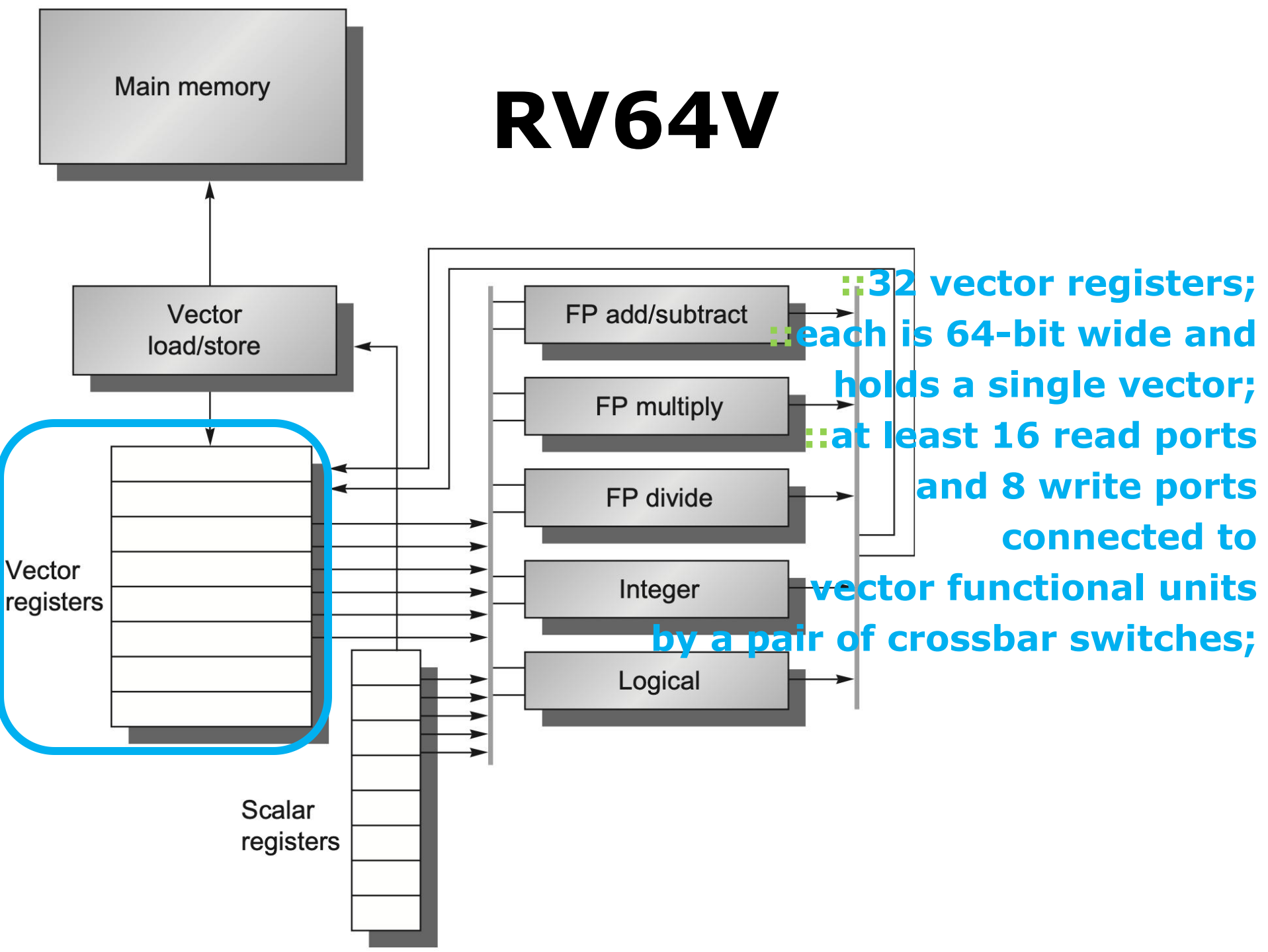
Vector Architecture

- Grab sets of data elements scattered about memory
- Place them in large sequential reg files
- Operate on data in those reg files
- Disperse the results back into memory
- A single instruction works on vector of data, which results in dozens of register-register operations on independent data elements

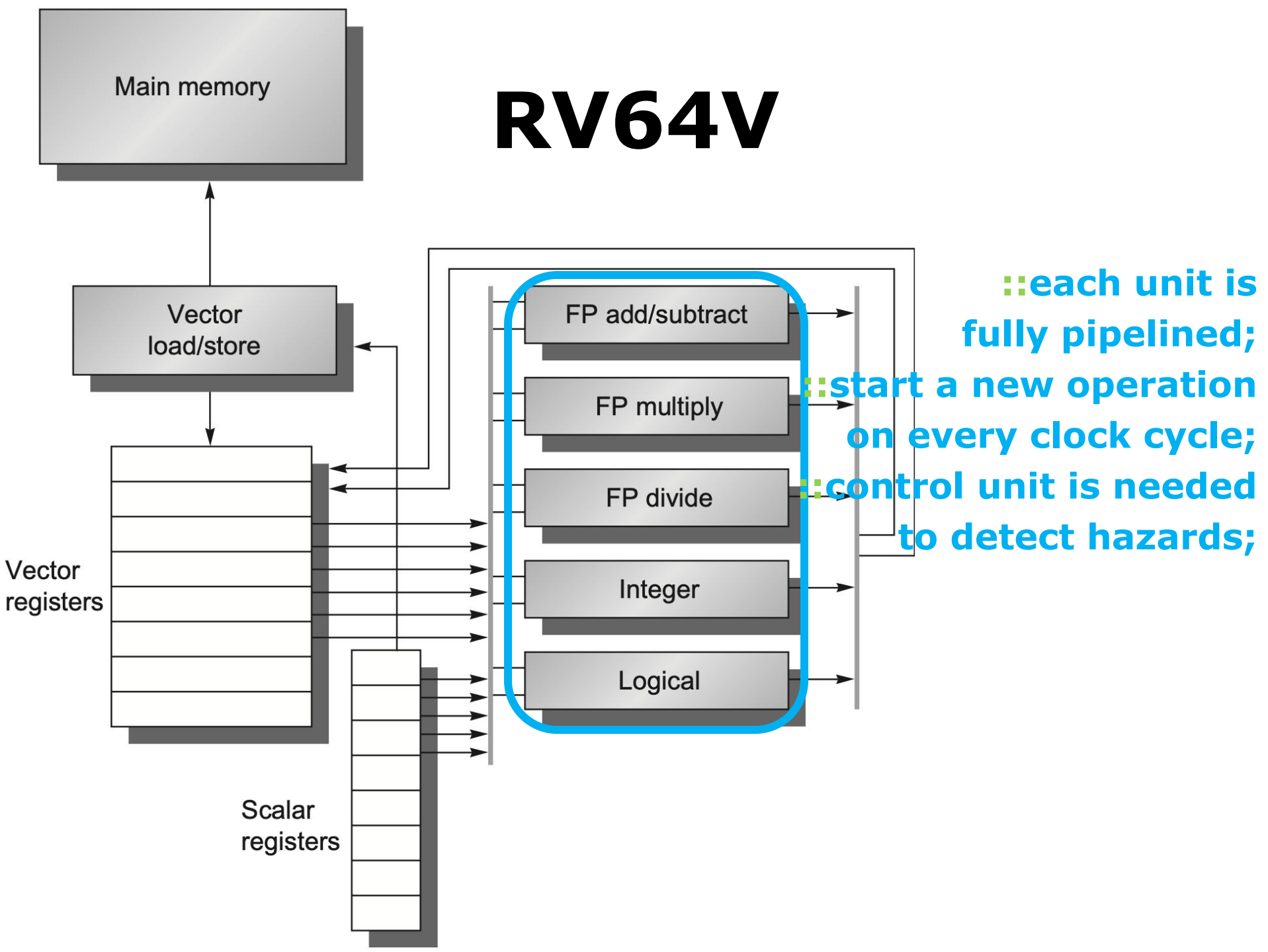
RV64V



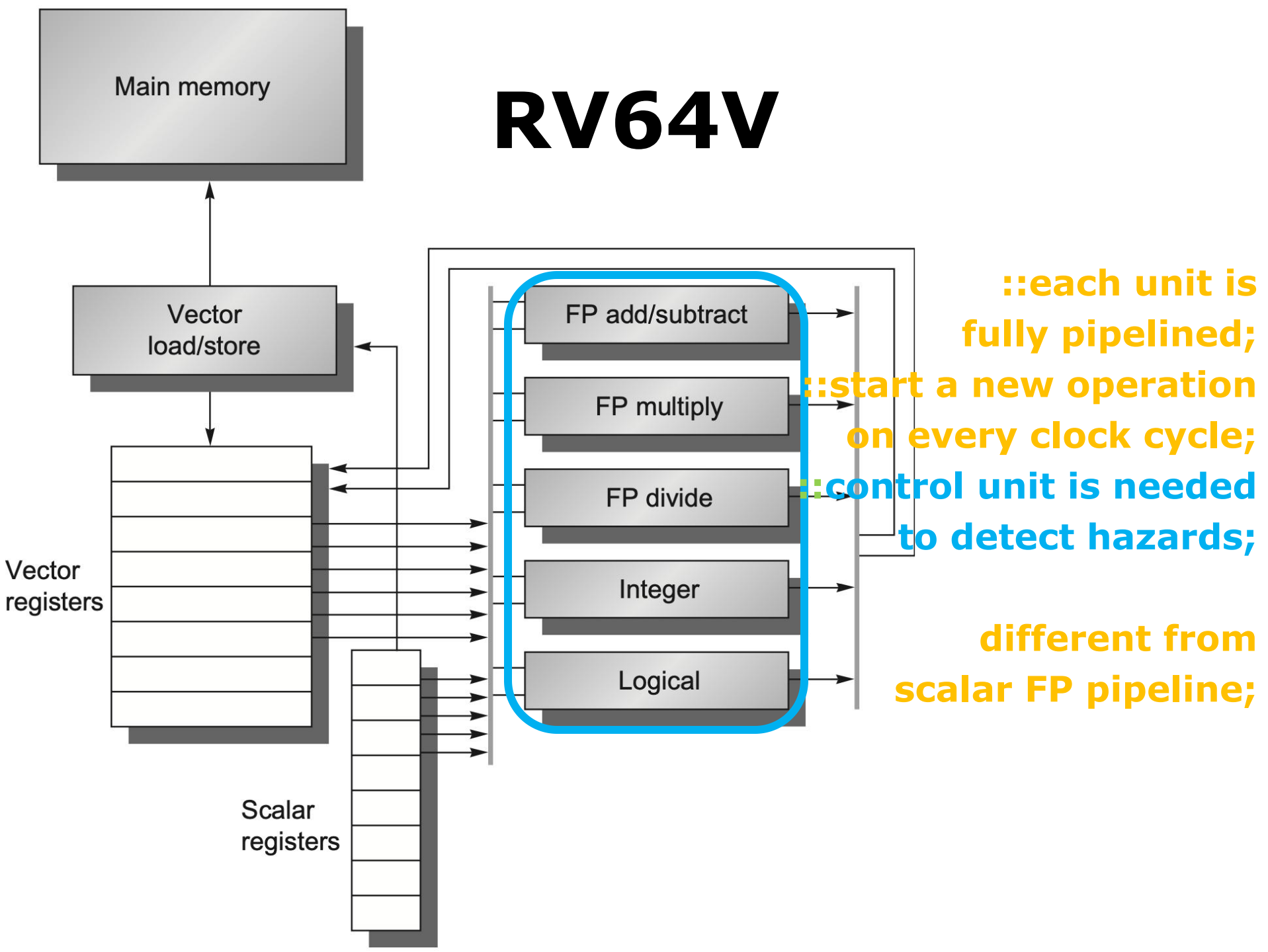
RV64V



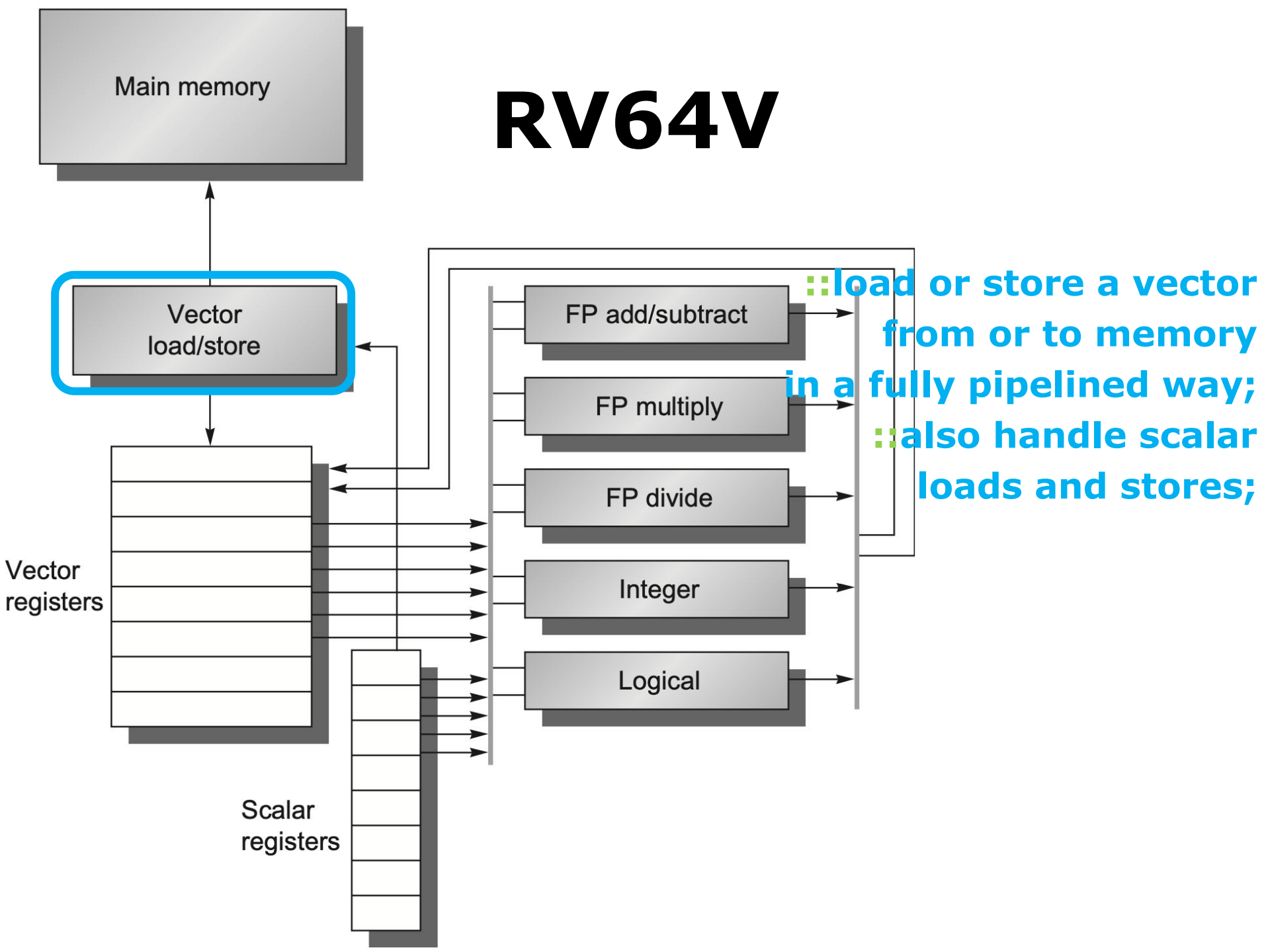
RV64V



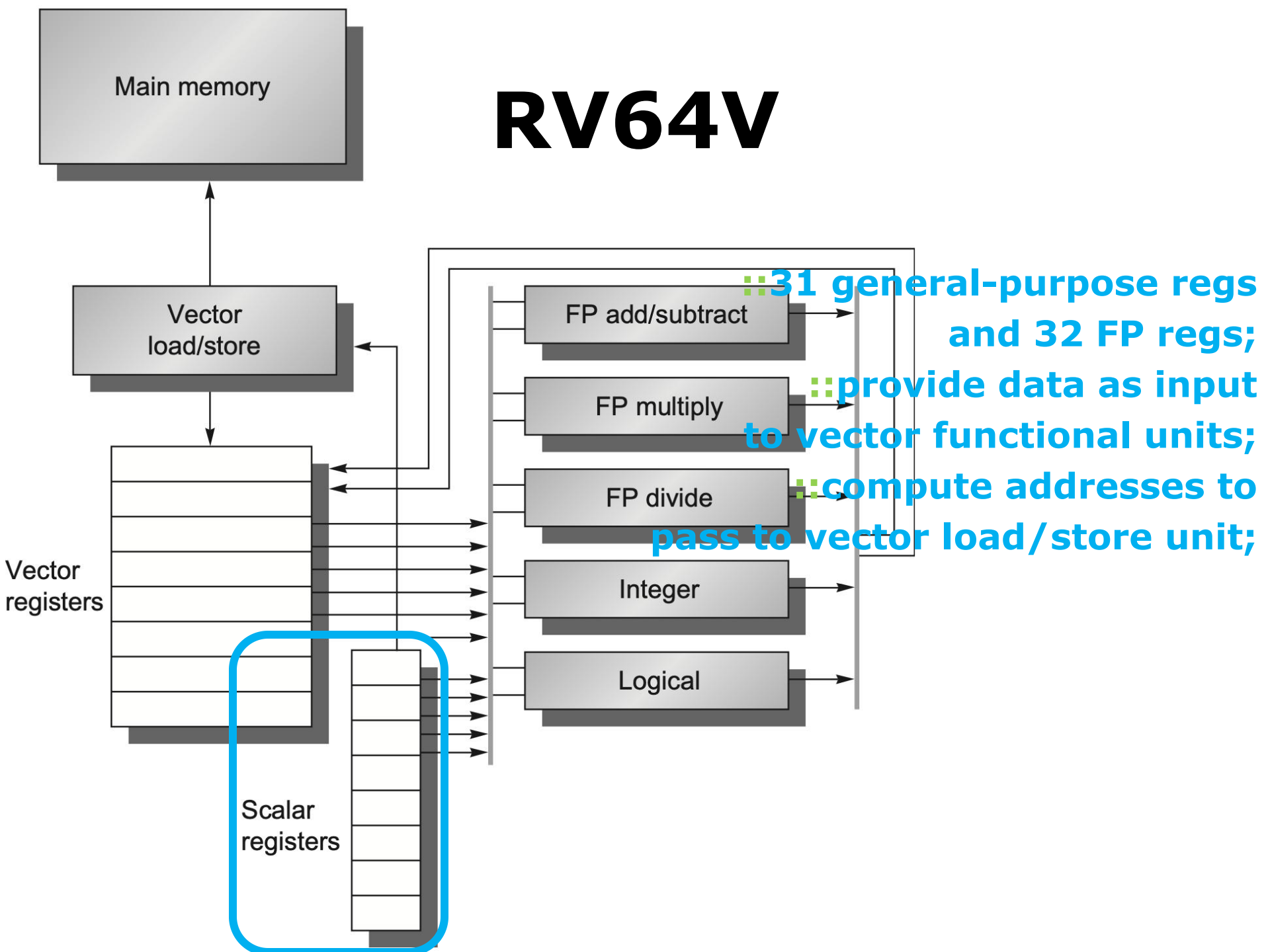
RV64V



RV64V



RV64V



RV64V Vector Instructions

Mnemonic	Name	Description
vadd	ADD	Add elements of V[rs1] and V[rs2], then put each result in V[rd]
vsub	SUBtract	Subtract elements of V[rs2] frpm V[rs1], then put each result in V[rd]
vmul	MULTiply	Multiply elements of V[rs1] and V[rs2], then put each result in V[rd]
vdiv	DIVide	Divide elements of V[rs1] by V[rs2], then put each result in V[rd]
vrem	REMAinder	Take remainder of elements of V[rs1] by V[rs2], then put each result in V[rd]
vsqrt	SQuare RooT	Take square root of elements of V[rs1], then put each result in V[rd]
vsll	Shift Left	Shift elements of V[rs1] left by V[rs2], then put each result in V[rd]
vsrl	Shift Right	Shift elements of V[rs1] right by V[rs2], then put each result in V[rd]
vsra	Shift Right Arithmetic	Shift elements of V[rs1] right by V[rs2] while extending sign bit, then put each result in V[rd]
vxor	XOR	Exclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vor	OR	Inclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vand	AND	Logical AND elements of V[rs1] and V[rs2], then put each result in V[rd]
vsgnj	SiGN source	Replace sign bits of V[rs1] with sign bits of V[rs2], then put each result in V[rd]
vsgnjn	Negative SiGN source	Replace sign bits of V[rs1] with complemented sign bits of V[rs2], then put each result in V[rd]
vsgnjx	Xor SiGN source	Replace sign bits of V[rs1] with xor of sign bits of V[rs1] and V[rs2], then put each result in V[rd]
vld	Load	Load vector register V[rd] from memory starting at address R[rs1]
vlds	Strided Load	Load V[rd] from address at R[rs1] with stride in R[rs2] (i.e., $R[rs1] + i \times R[rs2]$)
vldx	Indexed Load (Gather)	Load V[rs1] with vector whose elements are at $R[rs2] + V[rs2]$ (i.e., V[rs2] is an index)
vst	Store	Store vector register V[rd] into memory starting at address R[rs1]
vsts	Strided Store	Store V[rd] into memory at address R[rs1] with stride in R[rs2] (i.e., $R[rs1] + i \times R[rs2]$)
vstx	Indexed Store (Scatter)	Store V[rs1] into memory vector whose elements are at $R[rs2] + V[rs2]$ (i.e., V[rs2] is an index)
vpeq	Compare =	Compare elements of V[rs1] and V[rs2]. When equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpne	Compare !=	Compare elements of V[rs1] and V[rs2]. When not equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vplt	Compare <	Compare elements of V[rs1] and V[rs2]. When less than, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpxor	Predicate XOR	Exclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpor	Predicate OR	Inclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpand	Predicate AND	Logical AND 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
setvl	Set Vector Length	Set vl and the destination register to the smaller of mvl and the source regster

RV64V Vector

Instructions

Mnemonic	Name	Description
vadd	ADD	Add elements of V[rs1] and V[rs2], then put each result in V[rd]
vsub	SUBtract	Subtract elements of V[rs2] from V[rs1], then put each result in V[rd]
vmul	MULTiply	Multiply elements of V[rs1] and V[rs2], then put each result in V[rd]
vdiv	DIVide	Divide elements of V[rs1] by V[rs2], then put each result in V[rd]
vrem	REMAinder	Take remainder of elements of V[rs1] by V[rs2], then put each result in V[rd]
vsqrt	SQUare RooT	Take square root of elements of V[rs1], then put each result in V[rd]
vsll	Shift Left	Shift elements of V[rs1] left by V[rs2], then put each result in V[rd]
vsrl	Shift Right	Shift elements of V[rs1] right by V[rs2], then put each result in V[rd]
vsra	Shift Right Arithmetic	Shift elements of V[rs1] right by V[rs2] while extending sign bit, then put each result in V[rd]
vxor	XOR	Exclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vor	OR	Inclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vand	AND	Logical AND elements of V[rs1] and V[rs2], then put each result in V[rd]
vsgnj	SiGN source	Replace sign bits of V[rs1] with sign bits of V[rs2], then put each result in V[rd]
vsgnjn	Negative SiGN source	Replace sign bits of V[rs1] with complemented sign bits of V[rs2], then put each result in V[rd]
vsgnjx	Xor SiGN source	Replace sign bits of V[rs1] with xor of sign bits of V[rs1] and V[rs2], then put each result in V[rd]
vld	Load	Load vector register V[rd] from memory starting at address R[rs1]
vlds	Strided Load	Load V[rd] from address at R[rs1] with stride in R[rs2] (i.e., $R[rs1] + i \times R[rs2]$)
vldx	Indexed Load (Gather)	Load V[rs1] with vector whose elements are at $R[rs2] + V[rs2]$ (i.e., V[rs2] is an index)
vst	Store	Store vector register V[rd] into memory starting at address R[rs1]
vsts	Strided Store	Store V[rd] into memory at address R[rs1] with stride in R[rs2] (i.e., $R[rs1] + i \times R[rs2]$)
vstx	Indexed Store (Scatter)	Store V[rs1] into memory vector whose elements are at $R[rs2] + V[rs2]$ (i.e., V[rs2] is an index)
vpeq	Compare =	Compare elements of V[rs1] and V[rs2]. When equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpne	Compare !=	Compare elements of V[rs1] and V[rs2]. When not equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vplt	Compare <	Compare elements of V[rs1] and V[rs2]. When less than, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpxor	Predicate XOR	Exclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpor	Predicate OR	Inclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpand	Predicate AND	Logical AND 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
setvl	Set Vector Length	Set vl and the destination register to the smaller of mvl and the source register

all use R format;

suffix .vv for instructions

with both operands being vectors,

vs for vector and scalar,

.sv for scalar and vector;

commutative operations have

only the first two versions;

RV64V Vector

Instructions

data type and size?

Mnemonic	Name	Description
vadd	ADD	Add elements of V[rs1] and V[rs2], then put each result in V[rd]
vsub	SUBtract	Subtract elements of V[rs2] from V[rs1], then put each result in V[rd]
vmul	MULTiply	Multiply elements of V[rs1] and V[rs2], then put each result in V[rd]
vdiv	DIVide	Divide elements of V[rs1] by V[rs2], then put each result in V[rd]
vrem	REMAinder	Take remainder of elements of V[rs1] by V[rs2], then put each result in V[rd]
vsqrt	SQUare RooT	Take square root of elements of V[rs1], then put each result in V[rd]
vsll	Shift Left	Shift elements of V[rs1] left by V[rs2], then put each result in V[rd]
vsrl	Shift Right	Shift elements of V[rs1] right by V[rs2], then put each result in V[rd]
vsra	Shift Right Arithmetic	Shift elements of V[rs1] right by V[rs2] while extending sign bit, then put each result in V[rd]
vxor	XOR	Exclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vor	OR	Inclusive OR elements of V[rs1] and V[rs2], then put each result in V[rd]
vand	AND	Logical AND elements of V[rs1] and V[rs2], then put each result in V[rd]
vsgnj	SiGN source	Replace sign bits of V[rs1] with sign bits of V[rs2], then put each result in V[rd]
vsgnjn	Negative SiGN source	Replace sign bits of V[rs1] with complemented sign bits of V[rs2], then put each result in V[rd]
vsgnjx	Xor SiGN source	Replace sign bits of V[rs1] with xor of sign bits of V[rs1] and V[rs2], then put each result in V[rd]
vld	Load	Load vector register V[rd] from memory starting at address R[rs1]
vlds	Strided Load	Load V[rd] from address at R[rs1] with stride in R[rs2] (i.e., $R[rs1] + i \times R[rs2]$)
vldx	Indexed Load (Gather)	Load V[rs1] with vector whose elements are at $R[rs2] + V[rs2]$ (i.e., V[rs2] is an index)
vst	Store	Store vector register V[rd] into memory starting at address R[rs1]
vsts	Strided Store	Store V[rd] into memory at address R[rs1] with stride in R[rs2] (i.e., $R[rs1] + i \times R[rs2]$)
vstx	Indexed Store (Scatter)	Store V[rs1] into memory vector whose elements are at $R[rs2] + V[rs2]$ (i.e., V[rs2] is an index)
vpeq	Compare =	Compare elements of V[rs1] and V[rs2]. When equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpne	Compare !=	Compare elements of V[rs1] and V[rs2]. When not equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vplt	Compare <	Compare elements of V[rs1] and V[rs2]. When less than, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
vpxor	Predicate XOR	Exclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpor	Predicate OR	Inclusive OR 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
vpand	Predicate AND	Logical AND 1-bit elements of p[rs1] and p[rs2], then put each result in p[rd]
setvl	Set Vector Length	Set vl and the destination register to the smaller of mvl and the source register

Dynamic Register Typing

- Omitted in instruction encoding
- Associated a data type and data size with each vector register

Dynamic Register Typing

- Omitted in instruction encoding
- Associated a data type and data size with each vector register
- Before executing vector instructions, program configures the vector registers being used to specify their data type and widths:

Integer: 8, 16, 32, and 64 bits

Floating point: 16, 32, and 64 bits

Dynamic Register Typing

- Limit the number of instructions
(without using combinations of data types and sizes in instruction encoding)

Dynamic Register Typing

- Limit the number of instructions
- Enable programs to disable unused vector registers

example: 1024-byte vector memory,
4 vector registers are enabled,
then allocate $1024/4 = 256$ bytes for
each vector register (mvl: maximum
vector length)

Dynamic Register Typing

- Limit the number of instructions
- Enable programs to disable unused vector registers
- Support implicit conversions between different size operands depending on register configuration rather than additional explicit conversion instruction

how vector processor works?

DAXPY Example

- Double precision a x X plus Y
- X and Y have 32 elements
- x5 and x6 hold the starting addresses of X and Y, respectively

DAXPY Example

- RISC-V code:

```
      fld      f0,a           # Load scalar a
      addi     x28,x5,#256    # Last address to load
Loop: fld      f1,0(x5)       # Load X[i]
      fmul.d   f1,f1,f0       # a × X[i]
      fld      f2,0(x6)       # Load Y[i]
      fadd.d   f2,f2,f1       # a × X[i] + Y[i]
      fsd      f2,0(x6)       # Store into Y[i]
      addi     x5,x5,#8        # Increment index to X
      addi     x6,x6,#8        # Increment index to Y
      bne      x28,x5,Loop     # Check if done
```

DAXPY Example

- RV64V code:

```
vsetdcfg    4*FP64           # Enable 4 DP FP vregs
fld         f0,a             # Load scalar a
vld         v0,x5            # Load vector X
vmul        v1,v0,f0         # Vector-scalar mult
vld         v2,x6            # Load vector Y
vadd        v3,v1,v2         # Vector-vector add
vst         v3,x6            # Store the sum
vdisable    # Disable vector regs
```

- no loop-carried dependences
(dependences between iterations of a loop)

DAXPY Example

- RV64V code:

```
vsetdcfg    4*FP64          # Enable 4 DP FP vregs
fld         f0,a            # Load scalar a
vld         v0,x5           # Load vector X
vmul        v1,v0,f0        # Vector-scalar mult
vld         v2,x6           # Load vector Y
vadd        v3,v1,v2        # Vector-vector add
vst         v3,x6           # Store the sum
vdisable    # Disable vector regs
```

- assembler determines version of vector operations, e.g., vmul.vs and vadd.vv

DAXPY Example

- RV64V code:

```
vsetdcfg    4*FP64          # Enable 4 DP FP vregs
fld         f0,a            # Load scalar a
vld         v0,x5           # Load vector X
vmul        v1,v0,f0        # Vector-scalar mult
vld         v2,x6           # Load vector Y
vadd        v3,v1,v2        # Vector-vector add
vst         v3,x6           # Store the sum
vdisable    # Disable vector regs
```

- each vector instruction stalls only for the first element in each vector

DAXPY Example

- RV64V code: double-precision

```
vsetdcfg    4*FP64        # Enable 4 DP FP vregs
fld         f0,a          # Load scalar a
vld         v0,x5         # Load vector X
vmul        v1,v0,f0      # Vector-scalar mult
vld         v2,x6         # Load vector Y
vadd        v3,v1,v2      # Vector-vector add
vst         v3,x6         # Store the sum
vdisable    # Disable vector regs
```

- pipeline stalls are required only once per vector instruction, rather than once per vector element

DAXPY Example

- RV64V code: double-precision

```
vsetdcfg    4*FP64           # Enable 4 DP FP vregs
fld         f0,a             # Load scalar a
vld         v0,x5            # Load vector X
vmul        v1,v0,f0         # Vector-scalar mult
vld         v2,x6            # Load vector Y
vadd        v3,v1,v2         # Vector-vector add
vst         v3,x6            # Store the sum
vdisable    # Disable vector regs
```

- **Chaining:** forwarding of element-dependent operations, in that dependent operations are chained together

DAXPY Example

- RV64V code: double-precision

```
vsetdcfg    4*FP64        # Enable 4 DP FP vregs
fld         f0,a          # Load scalar a
vld         v0,x5         # Load vector X
vmul        v1,v0,f0      # Vector-scalar mult
vld         v2,x6         # Load vector Y
vadd        v3,v1,v2      # Vector-vector add
vst         v3,x6         # Store the sum
vdisable    # Disable vector regs
```

- **Chaining:** scalar and vector registers generate interim results which can be used immediately, without additional memory references

DAXPY Example

- RV64V code: double-precision

```
vsetdcfg    4*FP64        # Enable 4 DP FP vregs
fld         f0,a          # Load scalar a
vld         v0,x5         # Load vector X
vmul        v1,v0,f0      # Vector-scalar mult
vld         v2,x6         # Load vector Y
vadd        v3,v1,v2      # Vector-vector add
vst         v3,x6         # Store the sum
vdisable    # Disable vector regs
```

- **Flexible Chaining:** allow a vector instruction to chain to essentially any other active vector instruction

DAXPY Example

- RV64V code: single-precision X and a

```
vsetdcfg 1*FP32,3*FP64  # 1 32b, 3 64b vregs
flw      f0,a           # Load scalar a
vld      v0,x5          # Load vector X
vmul     v1,v0,f0       # Vector-scalar mult
vld      v2,x6          # Load vector Y
vadd     v3,v1,v2       # Vector-vector add
vst      v3,x6          # Store the sum
vdisable                # Disable vector regs
```

Vector Execution Time

- For a sequence of vector operations:
length of operand vectors?
structural hazards among operations?
data hazards?

Vector Execution Time

- For a single vector instruction:
vector length?
initiation rate? (the rate at which a vector unit consumes new operands and produces new results)

Vector Execution Time

- **Convoy:**

the set of vector instructions that could potentially execute together;
co-convoy instructions must not contain any structural hazards;
RAW is allowed in the same convoy;

- **Chime:**

the unit of time taken to execute the convoy

Vector Execution Time

- **Convoy:**
the set of vector instructions that could potentially execute together;
co-convoy instructions must not contain any structural hazards;
RAW is allowed in the same convoy;
- **Chime:**
the unit of time taken to execute the convoy
- **Cross-convoy instructions are serialized**

Vector Execution Time

- Example code sequence:

```
vld    v0,x5           # Load vector X
vmul   v1,v0,f0        # Vector-scalar multiply
vld    v2,x6           # Load vector Y
vadd   v3,v1,v2        # Vector-vector add
vst    v3,x6           # Store the sum
```

- Assume a single copy of each vector functional unit
- How many convoys/chimes?
- How many cycles per FLOP (floating-point operation)?

Vector Execution Time

- Example code sequence:



```
vld    v0,x5           # Load vector X
vmul   v1,v0,f0        # Vector-scalar multiply
vld    v2,x6           # Load vector Y
vadd   v3,v1,v2        # Vector-vector add
vst    v3,x6           # Store the sum
```

- How many convoys/chimes?

structural hazards: vld/vld, vld/vst

Vector Execution Time

- Example code sequence:



```
vld    v0,x5           # Load vector X
vmul   v1,v0,f0        # Vector-scalar multiply
vld    v2,x6           # Load vector Y
vadd   v3,v1,v2        # Vector-vector add
vst    v3,x6           # Store the sum
```

- How many convoys/chimes?: 3

structural hazards: vld/vld, vld/vst

convoy 1: vld, vmul (v0 dependence)

convoy 2: vld, vadd (v2 dependence)

convoy 3: vst

Vector Execution Time

- Example code sequence:

```
vld    v0,x5           # Load vector X
vmul   v1,v0,f0         # Vector-scalar multiply
vld    v2,x6           # Load vector Y
vadd   v3,v1,v2         # Vector-vector add
vst    v3,x6           # Store the sum
```

- How many cycles per FLOP?

assume 32-element vectors;

approximate 32 cycles per chime,

$32 \times 3 = 96$ cycles for 3 chimes,

$32 \times 2 = 64$ cycles for FLOPs [add,mul]

Vector Execution Time

- Example code sequence:

```
vld    v0,x5           # Load vector X
vmul   v1,v0,f0         # Vector-scalar multiply
vld    v2,x6           # Load vector Y
vadd   v3,v1,v2         # Vector-vector add
vst    v3,x6           # Store the sum
```

- How many cycles per FLOP?: 1.5
assume 32-element vectors;
approximate 32 cycles per chime,
 $96 / 64 = 1.5$ cycle per FLOP

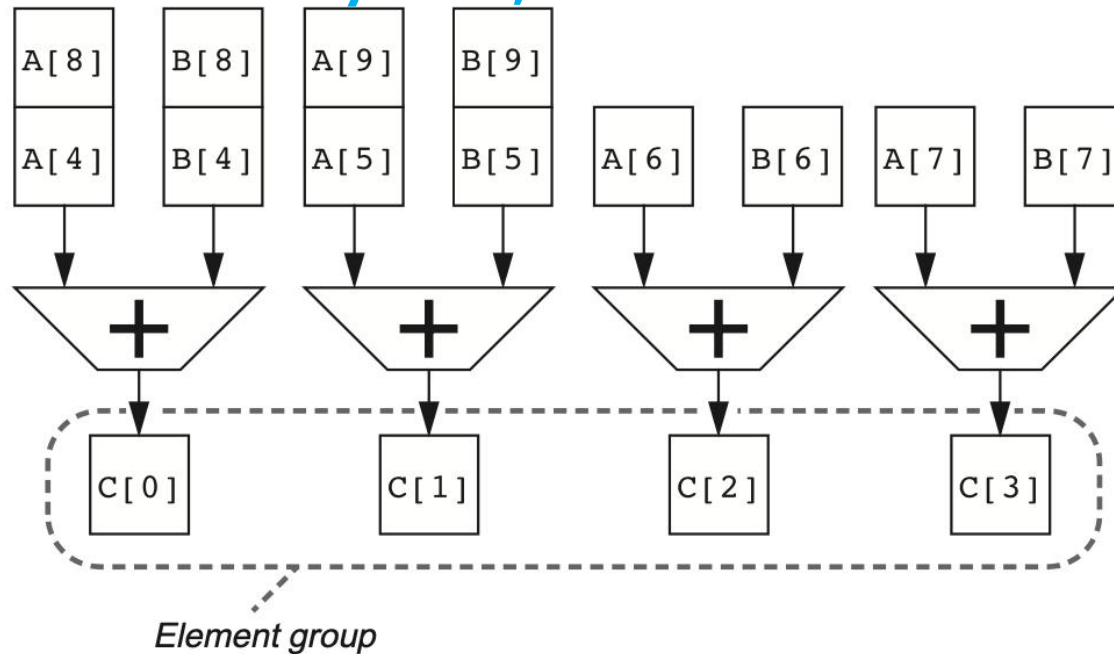
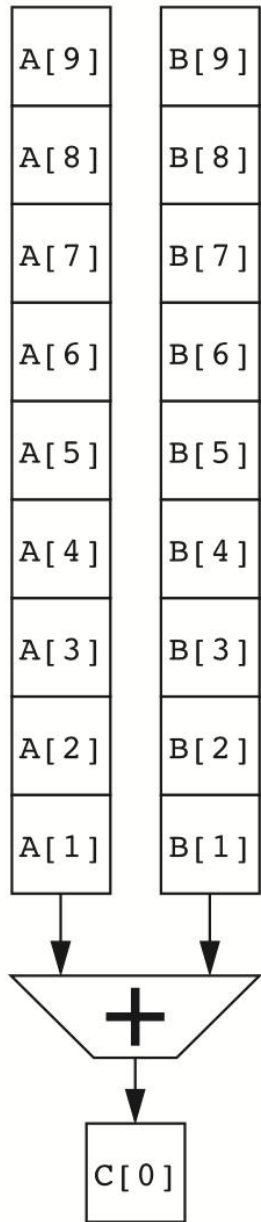
Vector Execution Time

- A vector sequence that consists of m convoys executes in m chimes
- For a vector length of n , this sequence takes approximately $m \times n$ clock cycles
- Ignore processor-specific overheads such as vector instruction issue overhead and vector start-up time (i.e., latency until the pipeline is full)

how to optimize vector arch?

Multiple Lanes

use multiple functional units to improve vector performance;
process several elements per clock cycle;



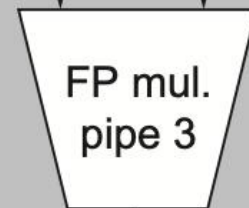
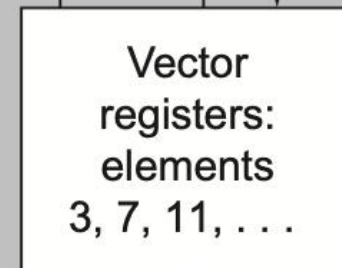
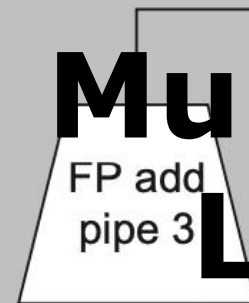
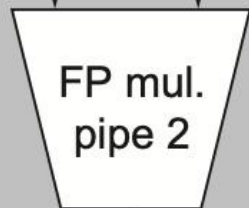
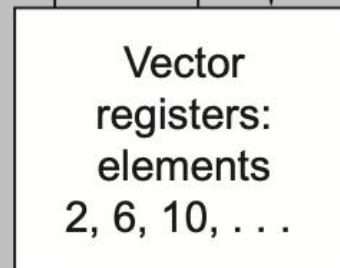
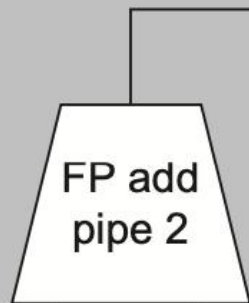
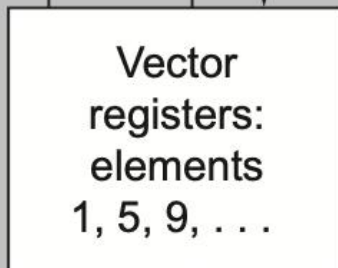
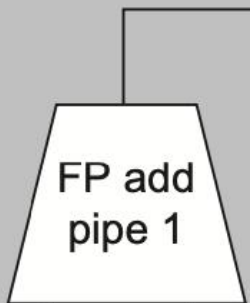
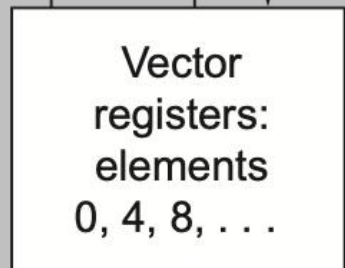
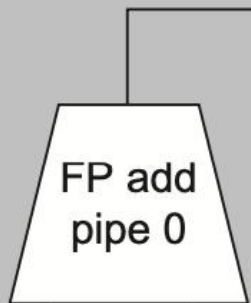
Lane 0

Lane 1

Lane 2

Lane 3

Multiple Lanes



Vector load-store unit

A long horizontal rectangular block at the bottom of the diagram, representing the shared vector load-store unit for all four lanes. Four vertical arrows point from the bottom of each lane's register block to this unit.

Lane 0

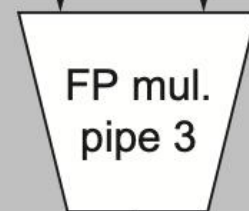
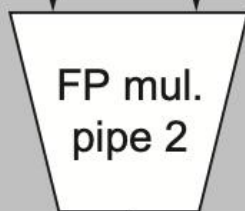
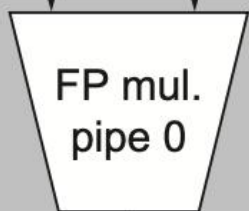
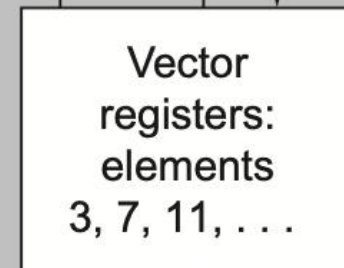
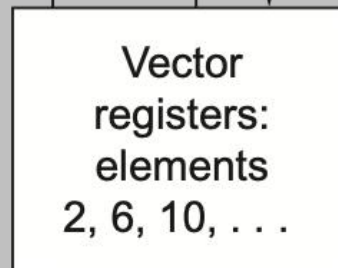
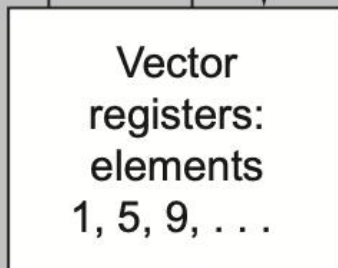
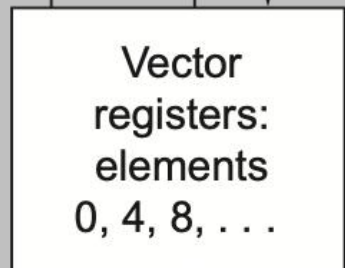
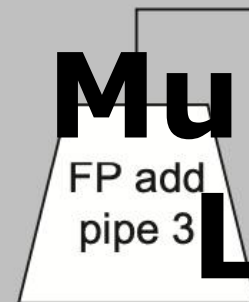
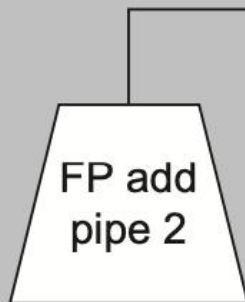
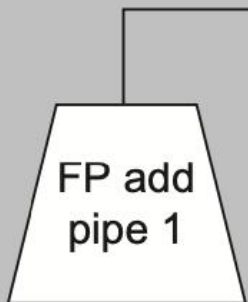
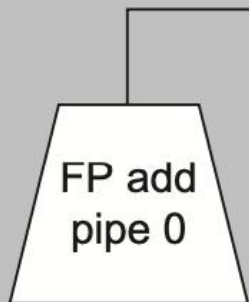
Lane 1

Lane 2

Lane 3

Multiple Lanes

vector register memory is divided across the lanes



Vector load-store unit

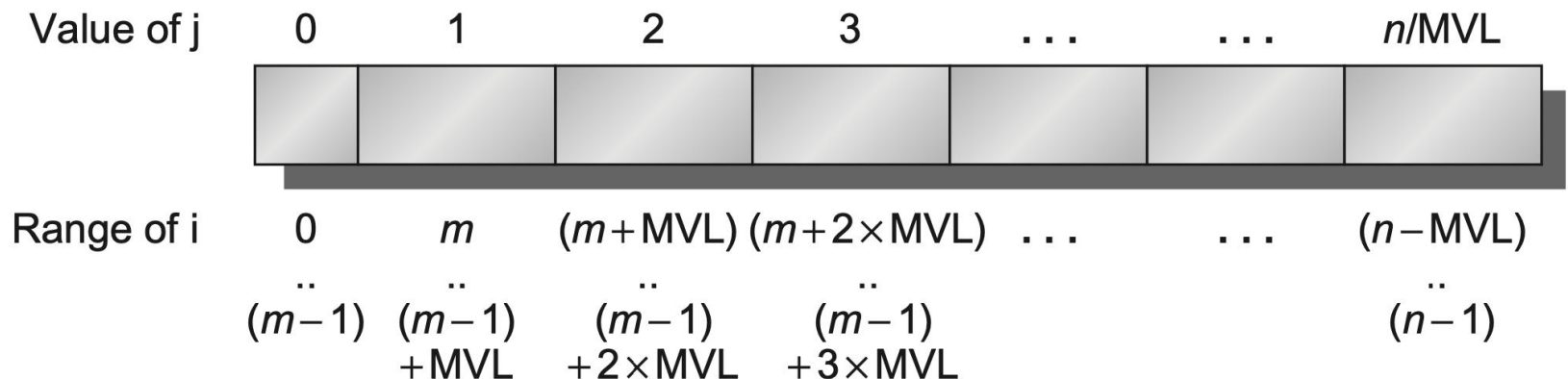
Vector-Length Registers

- Vector length is often unknown at compile time
- A single piece of code may require different vector lengths: n

```
for (i=0; i < n; i=i+1)  
    Y[i] = a * X[i] + Y[i];
```

Vector-Length Registers

- **Vector-length register (vl)** controls the length of any vector operation, including a vector load or store
- The value in vl cannot be greater than the maximum vector length (mvl)



Vector-Length Registers

- Strip mining

generate code such that each vector operation is done for a size $\leq \text{mvl}$: one loop handles any number of iterations that is a multiple of mvl , and another loop that handles any remaining iterations and must be less than mvl ;

- RISC-V instruction: `setvl`

$\text{vl} \leftarrow \min(\text{mvl}, n)$; n is loop variable

Vector-Length Registers

- RV64V code for vector DAXPY of any n

```
        vsetdcfg  2 DP FP      # Enable 2 64b Fl.Pt. registers
        fld       f0,a         # Load scalar a
loop:   setvl     t0,a0         # vl = t0 = min(mvl,n)
        vld       v0,x5        # Load vector X
        slli      t1,t0,3      # t1 = vl * 8 (in bytes)
        add       x5,x5,t1     # Increment pointer to X by vl*8
        vmul      v0,v0,f0     # Vector-scalar mult
        vld       v1,x6        # Load vector Y
        vadd      v1,v0,v1     # Vector-vector add
        sub       a0,a0,t0     # n -= vl (t0)
        vst       v1,x6        # Store the sum into Y
        add       x6,x6,t1     # Increment pointer to Y by vl*8
        bnez      a0,loop      # Repeat if n != 0
        vdisable                     # Disable vector regs
```

Vector-Length Registers

a0 initialized as vector size n;

- RV64V code for vector DAXPY of any n

```
    vsetdcfg  2 DP FP      # Enable 2 64b Fl.Pt. registers
    fld       f0,a         # Load scalar a
loop: setvl   t0,a0        # vl = t0 = min(mvl,n)
    vld       v0,x5        # Load vector X
    slli      t1,t0,3      # t1 = vl * 8 (in bytes)
    add       x5,x5,t1     # Increment pointer to X by vl*8
    vmul      v0,v0,f0     # Vector-scalar mult
    vld       v1,x6        # Load vector Y
    vadd      v1,v0,v1     # Vector-vector add
    sub       a0,a0,t0     # n -= vl (t0)
    vst       v1,x6        # Store the sum into Y
    add       x6,x6,t1     # Increment pointer to Y by vl*8
    bnez      a0,loop     # Repeat if n != 0
    vdisable             # Disable vector regs
```

Vector-Length Registers

DP = 64 bits = 8 bytes per element;

x5/x6 to track starting addresses of next vector segments;

- RV64V code for vector DAXPY of any n

```
    vsetdcfg 2 DP FP    # Enable 2 64b Fl.Pt. registers
    fld      f0,a        # Load scalar a
loop: setv1   t0,a0       # v1 = t0 = min(mv1,n)
    vld      v0,x5       # Load vector X
    slli     t1,t0,3     # t1 = v1 * 8 (in bytes)
    add      x5,x5,t1    # Increment pointer to X by v1*8
    vmul     v0,v0,f0    # Vector-scalar mult
    vld      v1,x6       # Load vector Y
    vadd     v1,v0,v1    # Vector-vector add
    sub      a0,a0,t0    # n -= v1 (t0)
    vst      v1,x6       # Store the sum into Y
    add      x6,x6,t1    # Increment pointer to Y by v1*8
    bnez     a0,loop     # Repeat if n != 0
    vdisable                # Disable vector regs
```

how to handle IF in loops?

```
for (i = 0; i < 64; i=i+1)
    if (X[i] != 0)
        X[i] = X[i] - Y[i];
```

how to handle IF in loops?

```
for (i = 0; i < 64; i=i+1)  
    if (X[i] != 0)  
        X[i] = X[i] - Y[i];
```

do not operate on elements if $X[i] \neq 0$

Predicate Registers

- **Vector-mask control**
use predicate registers to hold the mask and essentially provide conditional execution of each element operation
- When the predicate register p0 is set, all following vector instructions operate only on the vector elements whose corresponding entries in the predicate register is 1

Predicate Registers

- Vector-mask control
- When the predicate register p0 is set, all following vector instructions operate only on the vector elements whose corresponding entries in the predicate register is 1
- The entries in the destination vector register that correspond to 0 in the mask register are unaffected by the vector operation

Predicate Registers

- Vector-mask control
- When the predicate register p0 is set, all following vector instructions operate only on the vector elements whose corresponding entries in the predicate register is 1
- Enabling a predicate register initializes it to all 1s, meaning that subsequent vector instructions operate on all vector elements

Predicate Registers

- Example

vsetdcfg	2*FP64	# Enable 2 64b FP vector regs
vsetpcfgi	1	# Enable 1 predicate register
vld	v0,x5	# Load vector X into v0
vld	v1,x6	# Load vector Y into v1
fmv.d.x	f0,x0	# Put (FP) zero into f0
vpne	p0,v0,f0	# Set p0(i) to 1 if v0(i) != f0
vsub	v0,v0,v1	# Subtract under vector mask
vst	v0,x5	# Store the result in X
vdisable		# Disable vector registers
vpdisable		# Disable predicate registers

vpne	Compare !=	Compare elements of V[rs1] and V[rs2]. When not equal, put a 1 in the corresponding 1-bit element of p[rd]; otherwise, put 0
------	------------	--

Predicate Registers

- Example

```
vsetdcfg    2*FP64      # Enable 2 64b FP vector regs
vsetpcfgi   1           # Enable 1 predicate register
vld         v0,x5       # Load vector X into v0
vld         v1,x6       # Load vector Y into v1
fmv.d.x     f0,x0       # Put (FP) zero into f0
vpne        p0,v0,f0    # Set p0(i) to 1 if v0(i)!=f0
vsub        v0,v0,v1    # Subtract under vector mask
vst         v0,x5       # Store the result in X
vdisable    # Disable vector registers
vpdisable   # Disable predicate registers
```

- **IF-conversion:** for compiler writers to transform IF-stats to straight-line code

Memory Banks

- Support simultaneous accesses from multiple loads or stores (whose memory bank cycle time is usually several times larger than the processor cycle time)
- Load or store data words that are not sequential
- Share the same memory system by multiple processors

Memory Banks

- **Example**

Cray T90 with 32 processors, each with 4 loads and 2 stores per clock cycle;
processor cycle time: 2.167 ns;
SRAM cycle time: 15 ns;

- Question

minimum number of memory banks for all processors to run at the full memory bandwidth?

Memory Banks

- **Example**

Cray T90 with 32 processors, each with
4 loads and 2 stores per clock cycle;
processor cycle time: 2.167 ns;
SRAM cycle time: 15 ns;

- **Solution**

maximum number of memory

references per cycle: $(4+2) \times 32 = 192$;

SRAM cycle: $15/2.167 = 6.92 \approx 7$ cycles;

min number of banks: $192 \times 7 = 1344$;

Stride

- Memory positions of adjacent elements in a vector may not be sequential

```
for (i = 0; i < 100; i=i+1)
    for (j = 0; j < 100; j=j+1) {
        A[i][j] = 0.0;
        for (k = 0; k < 100; k=k+1)
            A[i][j] = A[i][j] + B[i][k] * D[k][j];
    }
```

- Assume row-major order, accesses to D-elements are separated by the row size times 8 (# of bytes per entry)

Stride

- **Stride:**
the distance separating elements to be gathered into a single vector register
- **Unit stride**
stride of 1, for adjacent elements
- **Nonunit stride**
stride greater than 1, for separated elements

Stride

- VLDS (load vector with stride)
instruction for fetching a nonunit stride vector into a vector register
- VSTS (store vector with stride)
instruction for storing a nonunit stride vector into memory

Stride

- **Example**

8 mem bank,
6-clock bank busy time,
12-cycle total memory latency

- Question

how long to complete a 64-element
vector with a stride of 1?
With a stride of 32?

Stride

- **Example**

8 mem bank,
6-clock bank busy time,
12-cycle total memory latency

- **Solution**

total memory latency =
memory access time + bank busy time;
fully pipelined vector initiation rate =
one word (element?) per clock cycle

Stride

- **Example**

8 mem bank,

6-clock bank busy time,

12-cycle total memory latency

- **Solution**

for a stride of 1, as bank # > bank busy time, no stalls due to bank conflicts (i.e., an access request reaches an occupied bank)

Stride

- **Example**

8 mem bank,
6-clock bank busy time,
12-cycle total memory latency

- **Solution**

for a stride of 1, as bank # > bank busy time, no stalls due to bank conflicts, after 12-cycle memory latency, one element per clock cycle:
 $12 + 64 = 76$ clock cycles

Stride

- **Example**

8 mem bank,

6-clock bank busy time,

12-cycle total memory latency

- **Solution**

for a stride of 32, as $\text{stride} \% \text{bank} = 0$,

every access request goes to bank 0

and causes a bank conflict and thus stalls

from the 2nd access:

$12 + 1 + 6 \times 63 = 391$ clock cycles

Stride

- A bank conflict and thus a stall occurs if:

$$\frac{\text{LCM}(\text{stride}, \text{number of banks})}{\text{stride}} < \text{bank busy time}$$

- Examples:
- $\text{LCM}(1, 8) / 1 = 8$? bank busy time
- $\text{LCM}(32, 8) / 32 = 1$? bank busy time
- $\text{LCM}(6, 16) / 6 = 8$? bank busy time
bank access: 0, 6, 12, 2, 8, 14, 4, 10,
0, 6, 12...

Stride

- A bank conflict and thus a stall occurs if:

$$\frac{\text{LCM}(\text{stride}, \text{number of banks})}{\text{stride}} < \text{bank busy time}$$

- Examples:

- $\text{LCM}(1, 8) / 1 = 8$? bank busy time
- $\text{LCM}(32, 8) / 32 = 1$? bank busy time
- $\text{LCM}(6, 16) / 6 = 8$? bank busy time

bank access: 0, 6, 12, 2, 8, 14, 4, 10,

0, 6, 12...

8 cycles to hit
the same bank

Gather-Scatter

- Sparse matrices using index vectors

```
for (i = 0; i < n; i=i+1)
    A[K[i]] = A[K[i]] + C[M[i]];
```

- Sparse vector/arrays A and C
- Index vectors K and M designate nonzero elements of A and C
- A and C have the same number of nonzero elements

Gather-Scatter

- Gather

fetch the vector whose elements are at the addresses given by adding a base address to the offsets given in the index vector;

- Scatter

store and expand the dense vector results from the result vector register to memory using the same index vector;

Gather-Scatter

- **vldx**
instruction to load vector indexed or gather
- **vstx**
instruction to store vector indexed or scatter

Gather-Scatter

- Sparse matrices using index vectors

```
for (i = 0; i < n; i=i+1)
    A[K[i]] = A[K[i]] + C[M[i]];
```

x5, x6: A, C

x7, x28: K, M

- RV64V code

vsetdcfg	4*FP64	# 4 64b FP vector registers
vld	v0, x7	# Load K[]
vldx	v1, x5, v0	# Load A[K[]]
vld	v2, x28	# Load M[]
vldx	v3, x6, v2	# Load C[M[]]
vadd	v1, v1, v3	# Add them
vstx	v1, x5, v0	# Store A[K[]]
vdisable		# Disable vector registers

how to vectorize program?

Programming Vector Arch

improve program using compiler hints to
vectorize more

Benchmark name	Operations executed in vector mode, compiler-optimized	Operations executed in vector mode, with programmer aid	Speedup from hint optimization
BDNA	96.1%	97.2%	1.52
MG3D	95.1%	94.5%	1.00
FLO52	91.5%	88.7%	N/A
ARC3D	91.1%	92.0%	1.01
SPEC77	90.3%	90.4%	1.07
MDG	87.7%	94.2%	1.49
TRFD	69.8%	73.7%	1.67
DYFESM	68.8%	65.6%	N/A
ADM	42.9%	59.6%	3.60
OCEAN	42.8%	91.2%	3.92
TRACK	14.4%	54.6%	2.52
SPICE	11.5%	79.9%	4.06
QCD	4.2%	75.1%	2.15

Data-Level Parallelism

example architectures:

Vector Architecture

Multimedia SIMD

GPU

Multimedia SIMD

- Suit for short, fixed-length vectors
256-bit-wide example operations:

Instruction category	Operands
Unsigned add/subtract	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Maximum/minimum	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Average	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Shift right/left	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Floating point	Sixteen 16-bit, eight 32-bit, four 64-bit, or two 128-bit

- **Omissions:** vector length register, strided or gather/scatter data transfer instructions, and mask registers

Multimedia SIMD

- RISC-V SIMD code for DAXPY:
256-bit SIMD multimedia instruction;
.4D operates on 4 DP operands at once

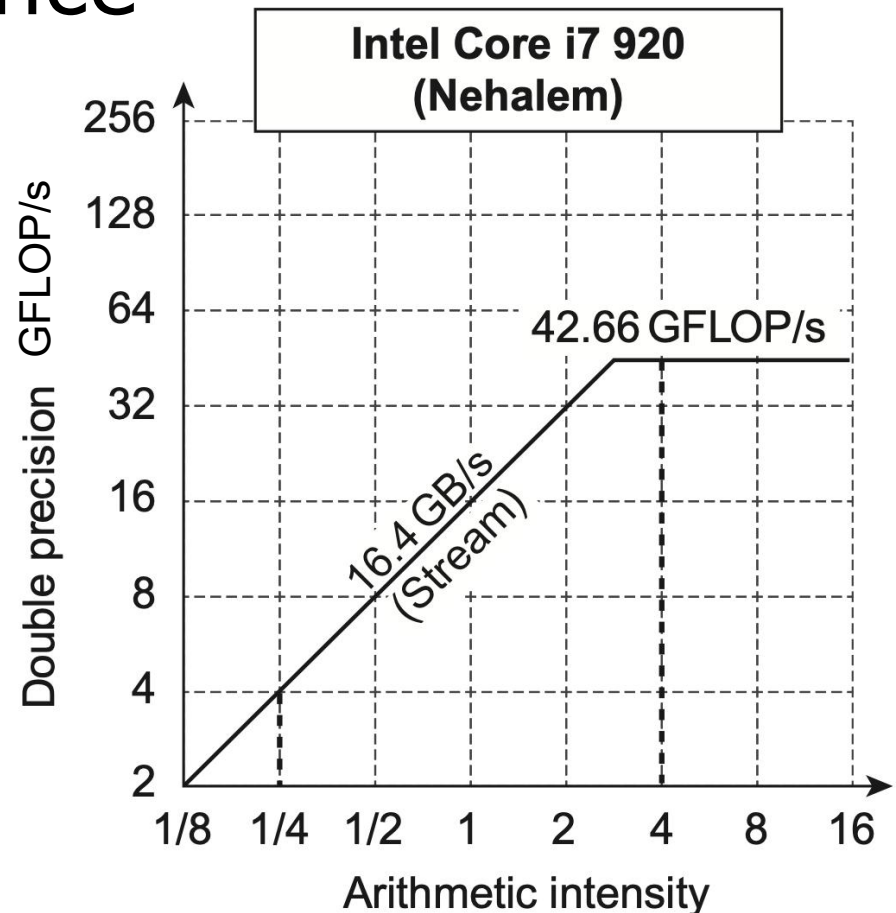
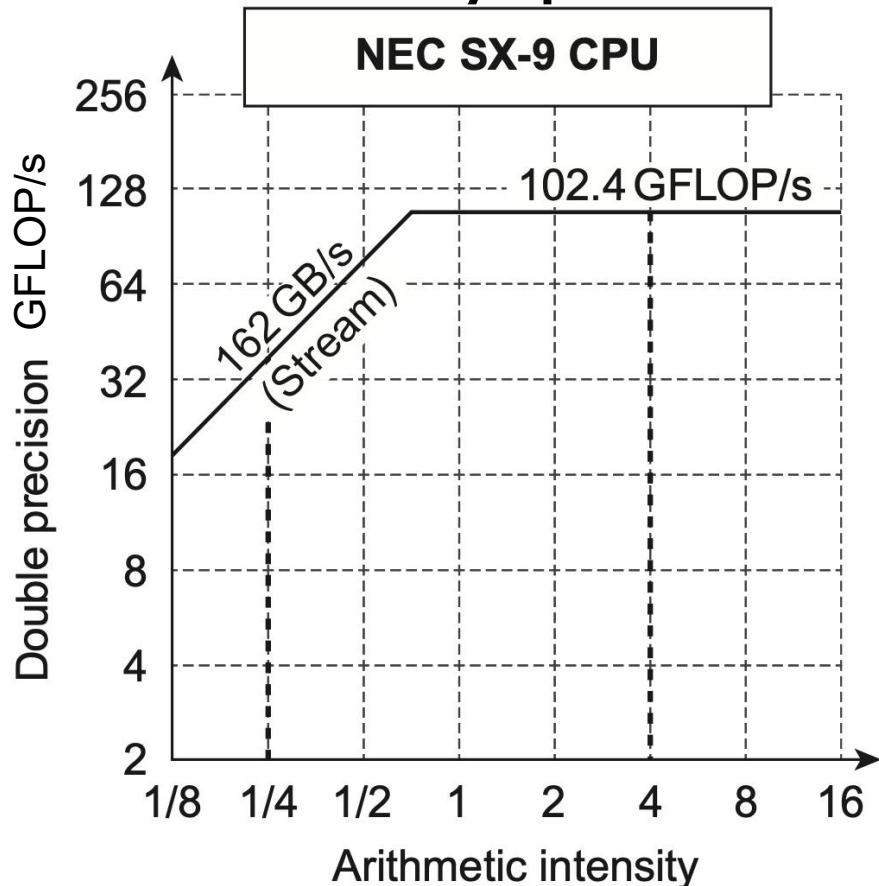
	<code>fld</code>	<code>f0, a</code>	<code>#Load scalar a</code>
	<code>splat.4D</code>	<code>f0, f0</code>	<code>#Make 4 copies of a</code>
	<code>addi</code>	<code>x28, x5, #256</code>	<code>#Last address to load</code>
Loop:	<code>fld.4D</code>	<code>f1, 0(x5)</code>	<code>#Load X[i] ... X[i+3]</code>
	<code>fmul.4D</code>	<code>f1, f1, f0</code>	<code>#a × X[i] ... a × X[i+3]</code>
	<code>fld.4D</code>	<code>f2, 0(x6)</code>	<code>#Load Y[i] ... Y[i+3]</code>
	<code>fadd.4D</code>	<code>f2, f2, f1</code>	<code># a × X[i] + Y[i] ...</code>
			<code># a × X[i+3] + Y[i+3]</code>
	<code>fsd.4D</code>	<code>f2, 0(x6)</code>	<code>#Store Y[i] ... Y[i+3]</code>
	<code>addi</code>	<code>x5, x5, #32</code>	<code>#Increment index to X</code>
	<code>addi</code>	<code>x6, x6, #32</code>	<code>#Increment index to Y</code>
	<code>bne</code>	<code>x28, x5, Loop</code>	<code>#Check if done</code>

Multimedia SIMD

- **Roofline Visual Performance Model**
correlate FP performance, memory performance, and arithmetic intensity in a two-dimensional graph
- **Arithmetic intensity**
ratio of FP operations per byte of memory accessed

Multimedia SIMD

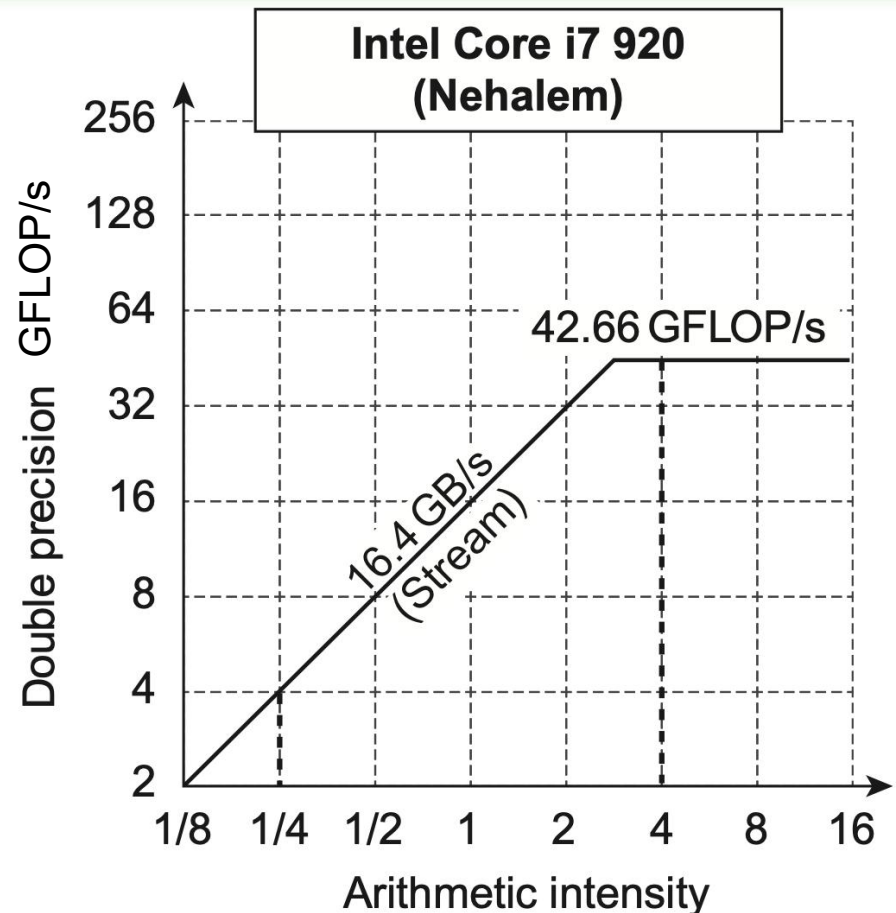
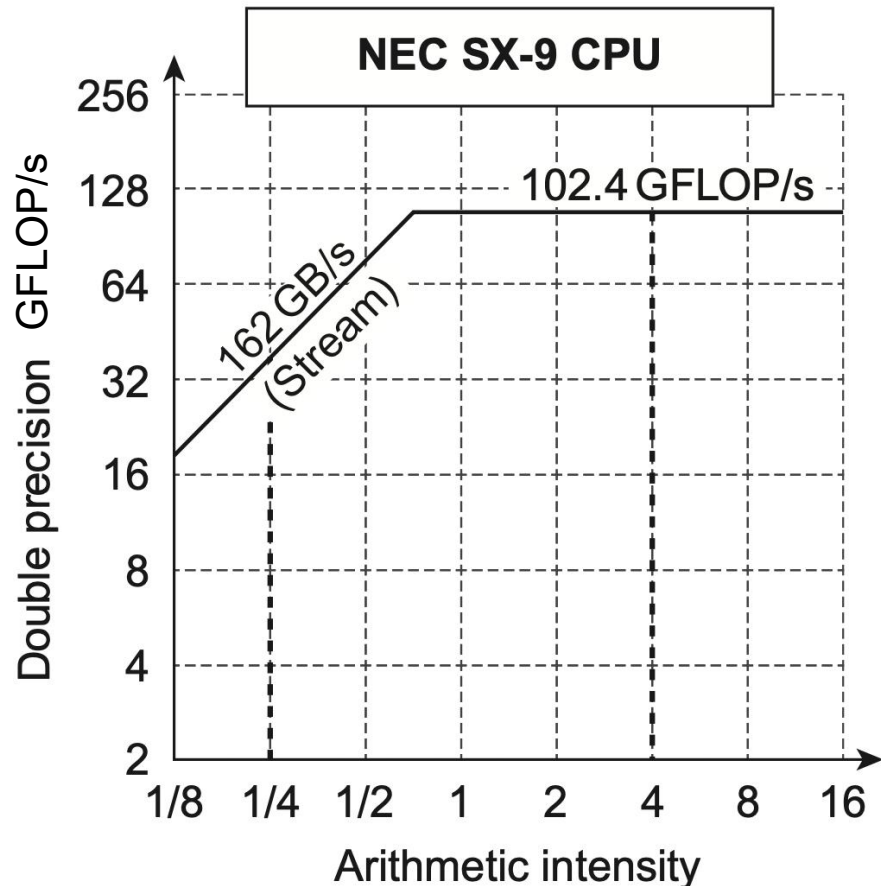
- Stream benchmark for finding peak memory performance



Multimedia SIMD

Attainable GFLOPs/s = Min(Peak Memory BW

× Arithmetic Intensity, Peak Floating – Point Perf.)



Data-Level Parallelism

example architectures:

Vector Architecture

Multimedia SIMD

GPU

GPU

- Support various types of parallelism: multithreading, MIMD, SIMD, ILP
- Challenge for programming:
not only getting good performance on GPU, but also
in coordinating the scheduling of computation on the system processor and the GPU
and the transfer of data between system memory and GPU memory

NVIDIA CUDA

- **C**ompute **U**nified **D**evice **A**rchitecture
- C/C++ for system processor (host)
- C/C++ dialect for GPU (device)
- Exploit parallelism via CUDA Threads:
SIMT – Single Instr, Multiple Thread;
Thread Block: threads blocked together
and executed in groups of threads;
multithreaded SIMD Processor;

NVIDIA CUDA

- **C**ompute **U**nified **D**evice **A**rchitecture
- C/C++ for system processor (**host**)
- C/C++ dialect for GPU (**device**)
- Exploit parallelism via CUDA Threads:
SIMT – Single Instr, Multiple Thread;
Thread Block: threads blocked together
and executed in groups of threads;
multithreaded SIMD Processor;

CUDA Program

- Function declaration:

host: `__host__`

device: `__device__`, `__global__`

- Variables declared with `__device__`:
allocated to GPU memory, accessible
by all multithreaded SIMD Processors

CUDA Program

- GPU function call:

name <<<dimGrid, dimblock>>>
(...parameter list...)

dimGrid: code dimensions (in Thread Blocks);
dimBlock: block dimensions (in Threads);

- Block identifier: blockIdx

- Thread identifier: threadIdx

- Numer of threads per block: blockDim
which comes from dimBlock

CUDA Program

- DAXPY in C:

```
// Invoke DAXPY
```

```
daxpy(n, 2.0, x, y);
```

```
// DAXPY in C
```

```
void daxpy(int n, double a, double *x, double *y)
```

```
{
```

```
    for (int i = 0; i < n; ++i)
```

```
        y[i] = a*x[i] + y[i];
```

```
}
```

CUDA Program

- DAXPY in CUDA:

```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
    daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__global__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
```

CUDA Program

GPU hardware handles parallel execution and thread management

- DAXPY in CUDA:

```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
    daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__global__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
```

CUDA Program

thread blocks execute independently, in any order
they cannot communicate directly

- DAXPY in CUDA:

```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
    daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__global__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
```


Type	Descriptive name	Closest old term outside of GPUs	Official CUDA/NVIDIA GPU term	Short explanation
Program abstractions	Vectorizable Loop	Vectorizable Loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more Thread Blocks (bodies of vectorized loop) that can execute in parallel
	Body of Vectorized Loop	Body of a (Strip-Mined) Vectorized Loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. They can communicate via local memory
	Sequence of SIMD Lane Operations	One iteration of a Scalar Loop	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask and predicate register
Machine object	A Thread of SIMD Instructions	Thread of Vector Instructions	Warp	A traditional thread, but it only contains SIMD instructions that are executed on a multithreaded SIMD Processor. Results stored depending on a per-element mask
	SIMD Instruction	Vector Instruction	PTX Instruction	A single SIMD instruction executed across SIMD Lanes
Processing hardware	Multithreaded SIMD Processor	(Multithreaded) Vector Processor	Streaming Multiprocessor	A multithreaded SIMD Processor executes threads of SIMD instructions, independent of other SIMD Processors
	Thread Block Scheduler	Scalar Processor	Giga Thread Engine	Assigns multiple Thread Blocks (bodies of vectorized loop) to multithreaded SIMD Processors
	SIMD Thread Scheduler	Thread Scheduler in a Multithreaded CPU	Warp Scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution
	SIMD Lane	Vector Lane	Thread Processor	A SIMD Lane executes the operations in a thread of SIMD instructions on a single element. Results stored depending on mask
Memory hardware	GPU Memory	Main Memory	Global Memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU
	Private Memory	Stack or Thread Local Storage (OS)	Local Memory	Portion of DRAM memory private to each SIMD Lane
	Local Memory	Local Memory	Shared Memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors
	SIMD Lane Registers	Vector Lane Registers	Thread Processor Registers	Registers in a single SIMD Lane allocated across a full Thread Block (body of vectorized loop)

Micro-architectural Features

Grid

Thread Blocks

Threads

Example:

vector multiply

8192 elements

$A = B \times C$

32 elements/thread;

16 threads/block;

16 blocks;

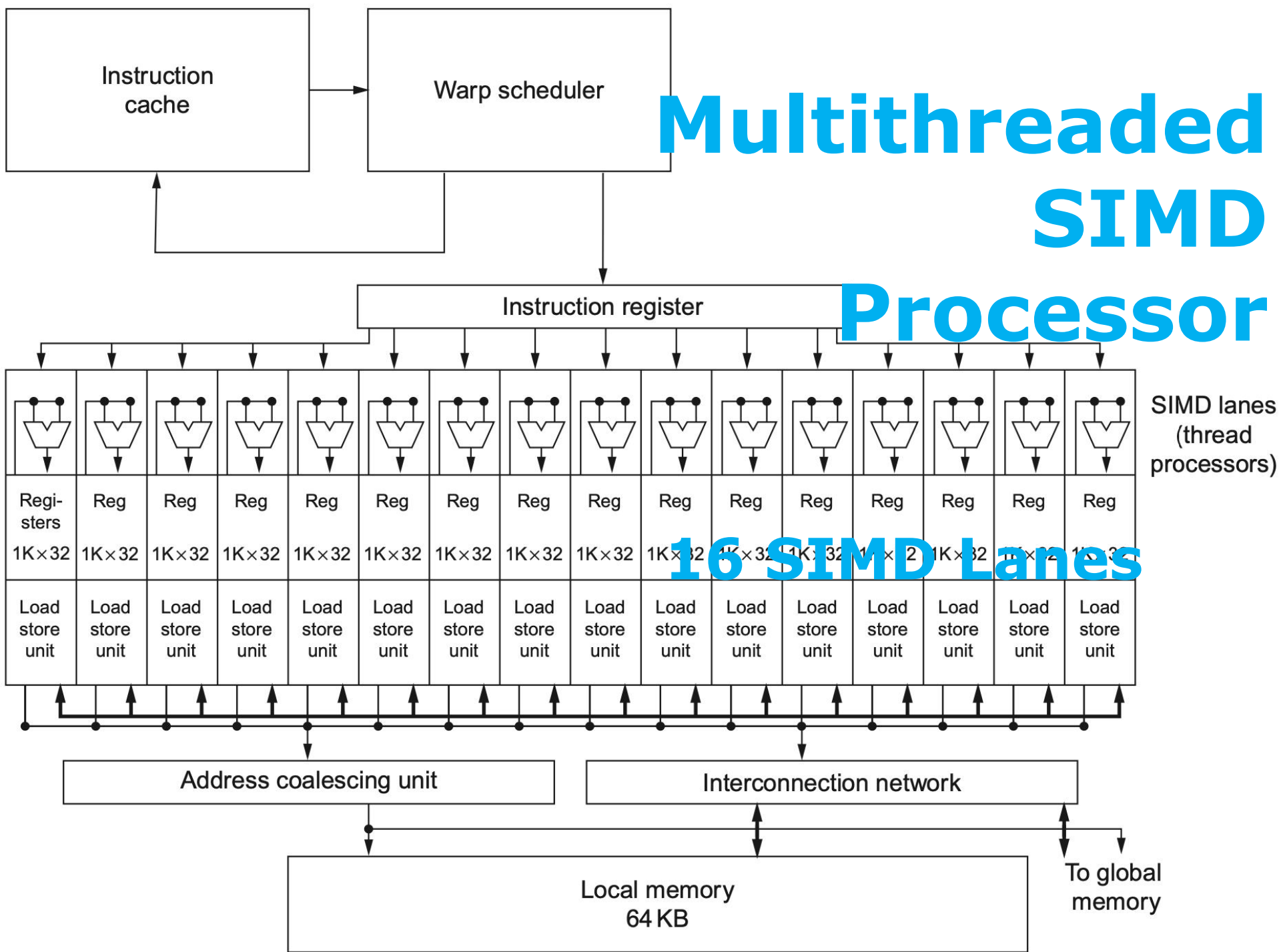
Thread Block 0	SIMD Thread0	A[0] = B [0] * C[0]
		A[1] = B [1] * C[1]
		
		A[31] = B [31] * C[31]
	SIMD Thread1	A[32] = B [32] * C[32]
		A[33] = B [33] * C[33]
		
		A[63] = B [63] * C[63]
	SIMD Thread15	A[64] = B [64] * C[64]
		
		A[479] = B [479] * C[479]
		
		A[480] = B [480] * C[480]
		A[481] = B [481] * C[481]
		
		A[511] = B [511] * C[511]

Grid

 A[7679] = B [7679] * C[7679]

Thread Block 15	SIMD Thread0	A[7680] = B [7680] * C[7680]
		A[7681] = B [7681] * C[7681]
		
		A[7711] = B [7711] * C[7711]
	SIMD Thread1	A[7712] = B [7712] * C[7712]
		A[7713] = B [7713] * C[7713]
		
		A[7743] = B [7743] * C[7743]
	SIMD Thread15	A[7744] = B [7744] * C[7744]
		
		A[8159] = B [8159] * C[8159]
		
		A[8160] = B [8160] * C[8160]
		A[8161] = B [8161] * C[8161]
		
		A[8191] = B [8191] * C[8191]

Multithreaded SIMD Processor



Pascal P100 GPU

56 Multithreaded SIMD Processor



SIMD Thread Scheduler

- Know which threads of SIMD instructions are ready to run
- Send them off to a dispatch unit to be run on the multithreaded SIMD Processor

SIMD Thread Scheduler

- Two-level GPU hardware schedulers:
- Thread Block Scheduler
assign thread blocks (bodies of vectorized loops) to multithreaded SIMD Processors;
- SIMD Thread Scheduler
schedule when threads of SIMD instructions within a SIMD Processor should run

NVIDIA GPU ISA

- **PTX: Parallel Thread Execution**
provide a stable instruction set for compilers as well as compatibility across generations of GPUs
- PTX instructions translated to an internal form in software and load time on a GPU

NVIDIA GPU ISA

- PTX instruction format:
opcode.type d, a, b, c.
d: destination operand
a, b, c: source operands
- Operation types:

Type	.type specifier
Untyped bits 8, 16, 32, and 64 bits	.b8, .b16, .b32, .b64
Unsigned integer 8, 16, 32, and 64 bits	.u8, .u16, .u32, .u64
Signed integer 8, 16, 32, and 64 bits	.s8, .s16, .s32, .s64
Floating Point 16, 32, and 64 bits	.f16, .f32, .f64

NVIDIA GPU ISA

- PTX instruction format:
opcode.type d, a, b, c.
d: destination operand
a, b, c: source operands
- Source operands:
32-bit / 64-bit registers, constant
- Destination operands:
registers for non-store instructions,
memory addresses for store instructions

PTX Instructions

Group	Instruction	Example	Meaning	Comments
Arithmetic	arithmetic .type = .s32, .u32, .f32, .s64, .u64, .f64			
	add.type	add.f32 d, a, b	$d = a + b;$	
	sub.type	sub.f32 d, a, b	$d = a - b;$	
	mul.type	mul.f32 d, a, b	$d = a * b;$	
	mad.type	mad.f32 d, a, b, c	$d = a * b + c;$	multiply-add
	div.type	div.f32 d, a, b	$d = a / b;$	multiple microinstructions
	rem.type	rem.u32 d, a, b	$d = a \% b;$	integer remainder
	abs.type	abs.f32 d, a	$d = a ;$	
	neg.type	neg.f32 d, a	$d = 0 - a;$	
	min.type	min.f32 d, a, b	$d = (a < b) ? a : b;$	floating selects non-NaN
	max.type	max.f32 d, a, b	$d = (a > b) ? a : b;$	floating selects non-NaN
	setp.cmp.type	setp.lt.f32 p, a, b	$p = (a < b);$	compare and set predicate
	numeric .cmp = eq, ne, lt, le, gt, ge; unordered cmp = equ, neu, ltu, leu, gtu, geu, num, nan			
	mov.type	mov.b32 d, a	$d = a;$	move
	selp.type	selp.f32 d, a, b, p	$d = p ? a : b;$	select with predicate
	cvt.dtype.atype	cvt.f32.s32 d, a	$d = \text{convert}(a);$	convert atype to dtype

PTX Instructions

Group	Instruction	Example	Meaning	Comments
Special function	special.type = .f32 (some .f64)			
	rcp.type	rcp.f32 d, a	$d = 1/a;$	reciprocal
	sqr.type	sqr.f32 d, a	$d = \sqrt{a};$	square root
	rsqr.type	rsqr.f32 d, a	$d = 1/\sqrt{a};$	reciprocal square root
	sin.type	sin.f32 d, a	$d = \sin(a);$	sine
	cos.type	cos.f32 d, a	$d = \cos(a);$	cosine
	lg2.type	lg2.f32 d, a	$d = \log(a)/\log(2)$	binary logarithm
	ex2.type	ex2.f32 d, a	$d = 2^{**} a;$	binary exponential
Logical	logic.type = .pred, .b32, .b64			
	and.type	and.b32 d, a, b	$d = a \& b;$	
	or.type	or.b32 d, a, b	$d = a b;$	
	xor.type	xor.b32 d, a, b	$d = a \wedge b;$	
	not.type	not.b32 d, a, b	$d = \sim a;$	one's complement
	cnot.type	cnot.b32 d, a, b	$d = (a == 0) ? 1 : 0;$	C logical not
	shl.type	shl.b32 d, a, b	$d = a \ll b;$	shift left
	shr.type	shr.s32 d, a, b	$d = a \gg b;$	shift right

PTX Instructions

Group	Instruction	Example	Meaning	Comments
Memory access	memory.space = .global, .shared, .local, .const; .type = .b8, .u8, .s8, .b16, .b32, .b64			
	ld.space.type	ld.global.b32 d, [a+off]	d = *(a+off);	load from memory space
	st.space.type	st.shared.b32 [d+off], a	*(d+off) = a;	store to memory space
	tex.nd.dtyp.btype	tex.2d.v4.f32.f32 d, a, b	d = tex2d(a, b);	texture lookup
	atom.spc.op.type	atom.global.add.u32 d,[a], b atom.global.cas.b32 d,[a], b, c	atomic { d = *a; *a = op(*a, b); }	atomic read-modify-write operation
	atom.op = and, or, xor, add, min, max, exch, cas; .spc = .global; .type = .b32			
Control flow	branch	@p bra target	if (p) goto target;	conditional branch
	call	call (ret), func, (params)	ret = func(params);	call function
	ret	ret	return;	return from function call
	bar.sync	bar.sync d	wait for threads	barrier synchronization
	exit	exit	exit;	terminate thread execution

PTX Instructions

- DAXPY in PTX:

```
shl.u32 R8, blockIdx, 8    ; Thread Block ID * Block size
                           ; (256 or 28)
add.u32 R8, R8, threadIdx ; R8 = i = my CUDA Thread ID
shl.u32 R8, R8, 3          ; byte offset
ld.global.f64 RD0, [X+R8] ; RD0 = X[i]
ld.global.f64 RD2, [Y+R8] ; RD2 = Y[i]
mul.f64 RD0, RD0, RD4      ; Product in RD0 = RD0 * RD4
                           ; (scalar a)
add.f64 RD0, RD0, RD2      ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0 ; Y[i] = sum (X[i]*a + Y[i])
```

PTX Instructions

1 thread per element, 256 threads per block,
compute thread index

- DAXPY in PTX:

```
shl.u32 R8, blockIdx, 8    ; Thread Block ID * Block size  
                           ; (256 or  $2^8$ )  
add.u32 R8, R8, threadIdx ; R8 = i = my CUDA Thread ID  
shl.u32 R8, R8, 3          ; byte offset  
ld.global.f64 RD0, [X+R8] ; RD0 = X[i]  
ld.global.f64 RD2, [Y+R8] ; RD2 = Y[i]  
mul.f64 RD0, RD0, RD4      ; Product in RD0 = RD0 * RD4  
                           ; (scalar a)  
add.f64 RD0, RD0, RD2      ; Sum in RD0 = RD0 + RD2 (Y[i])  
st.global.f64 [Y+R8], RD0 ; Y[i] = sum (X[i]*a + Y[i])
```

PTX Instructions

each element is 64-bit = 8-byte,
compute address index

- DAXPY in PTX:

```
shl.u32 R8, blockIdx, 8    ; Thread Block ID * Block size  
                           ; (256 or  $2^8$ )
```

```
add.u32 R8, R8, threadIdx ; R8 = i = my CUDA Thread ID
```

```
shl.u32 R8, R8, 3          ; byte offset
```

```
ld.global.f64 RD0, [X+R8]; RD0 = X[i]
```

```
ld.global.f64 RD2, [Y+R8]; RD2 = Y[i]
```

```
mul.f64 RD0, RD0, RD4      ; Product in RD0 = RD0 * RD4  
                           ; (scalar a)
```

```
add.f64 RD0, RD0, RD2      ; Sum in RD0 = RD0 + RD2 (Y[i])
```

```
st.global.f64 [Y+R8], RD0; Y[i] = sum (X[i]*a + Y[i])
```

Conditional Branch

- GPU branch hardware
 - predicate registers
 - internal masks
 - branch synchronization stack
 - instruction markers

Predicate Register

- Specified by programmer at PTX assembler level
- Individual per-thread-lane predication of each instruction
- 1-bit predicate registers
- GPU set predication instruction: `setp`

Predicate Register

- SIMD instructions in the threads inside THEN part of IF statement broadcast operations to all SIMD lanes
- Lanes with predicate set to 1 perform an operation or store a result; while other lanes do not
- ELSE-instructions use the complement of the predicate

Predicate Register

- For equal-length paths, IF-THEN-ELSE operates at 50% efficiency or less
- Doubly nested IF statements with equal-length paths run at 25% efficiency

Branch Sync Stack

- Each SIMD Thread has its own stack at GPU hardware instruction level
- **Stack entry:**
 - identifier token
 - target instruction address
 - target thread-active mask

Branch Sync Stack

- Special instructions that push stack entries for a SIMD Thread
- Special instructions and instruction markers that pop a stack entry or unwind the stack to a specified entry and branch to the target instruction address with the target thread-active mask

Conditional Branch

- Optimize a simple outer-level IF-THEN-ELSE statement coded with PTX branch instructions to solely predicated GPU instructions

Conditional Branch

- Optimize more complex control flow to a mixture of predication and GPU branch instructions with special instructions and markers that use the branch sync stack
- Branch **diverges** when some lanes branch to the target address and push a stack entry
- Branch **converges** when lanes pop stack entry & branch to stack-entry address

Conditional Branch

- Conditional statement

```
if (X[i] != 0)
    X[i] = X[i] - Y[i];
else X[i] = Z[i];
```

- PTX instructions

```
ld.global.f64 RD0, [X+R8]          ; RD0 = X[i]
setp.neq.s32 P1, RD0, #0           ; P1 is predicate reg 1
@!P1, bra ELSE1, *Push             ; Push old mask, set new
                                   ; mask bits if P1 false, go to ELSE1
ld.global.f64 RD2, [Y+R8]          ; RD2 = Y[i]
sub.f64 RD0, RD0, RD2              ; Difference in RD0
st.global.f64 [X+R8], RD0          ; X[i] = RD0
@P1, bra ENDIF1, *Comp             ; complement mask bits
                                   ; if P1 true, go to ENDIF1
ELSE1: ld.global.f64 RD0, [Z+R8]    ; RD0 = Z[i]
      st.global.f64 [X+R8], RD0    ; X[i] = RD0
ENDIF1: <next instruction>, *Pop    ; pop to restore old mask
```

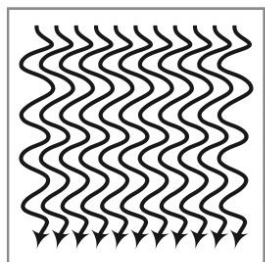

CUDA thread



Per-CUDA thread private memory

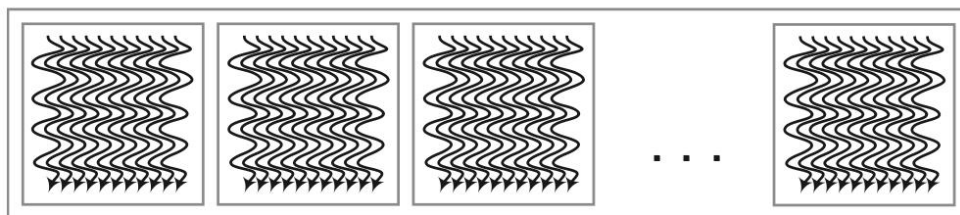
GPU Memory

Thread block



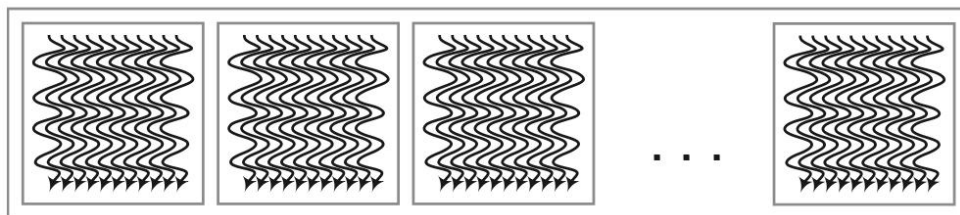
Per-block
local memory

Grid 0



Inter-grid synchronization

Grid 1



Sequence

GPU memory

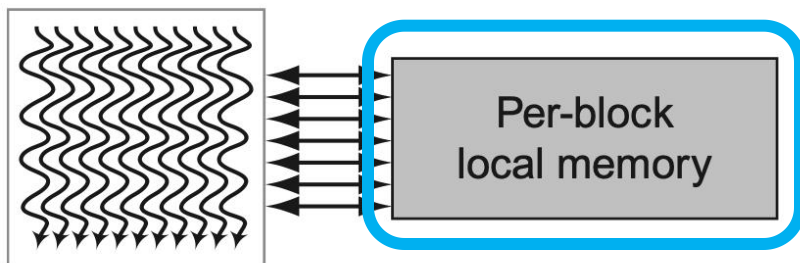


CUDA thread

GPU Memory

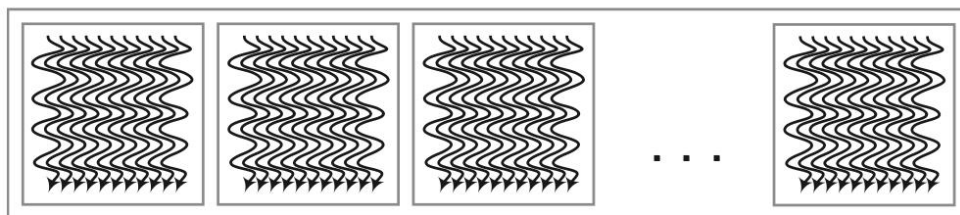


Thread block

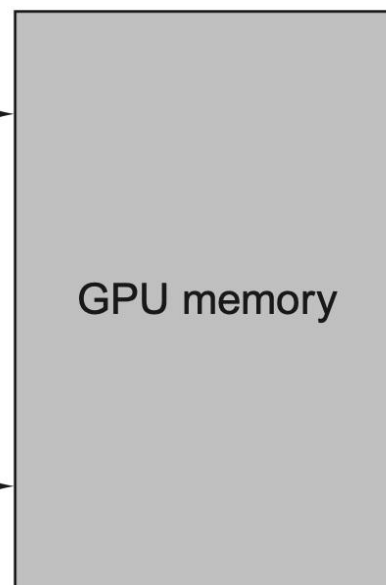


shared by co-block
threads;
private to other threads
and host;

Grid 0

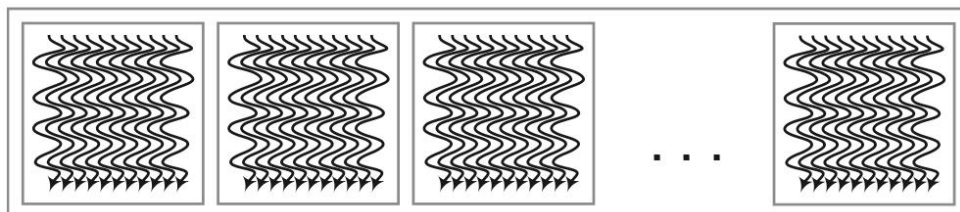


Sequence



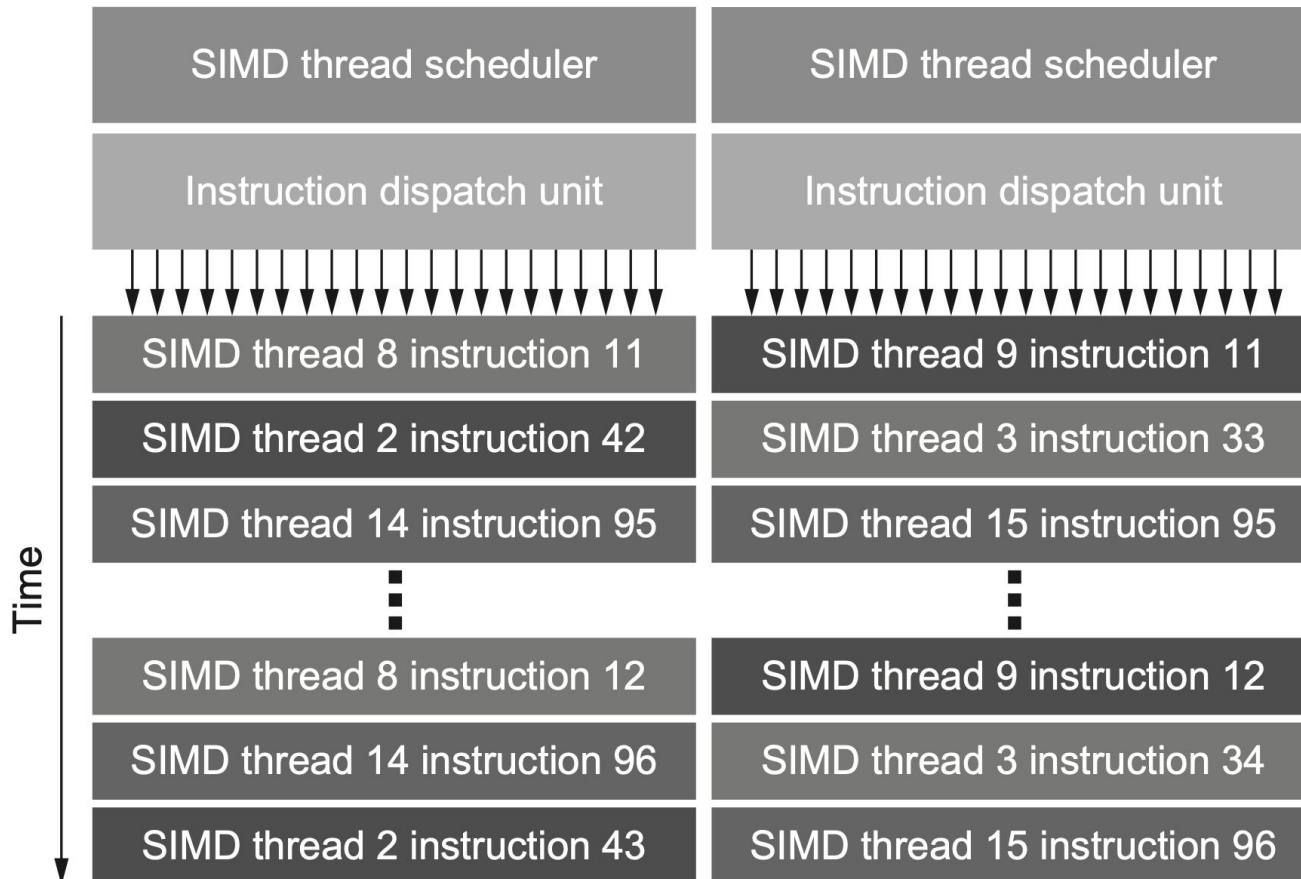
Inter-grid synchronization

Grid 1



Pascal GPU

- Dual SIMD thread scheduler schedules two instructions each clock cycle



Instruction Cache

Instruction Buffer

SIMD Thread Scheduler

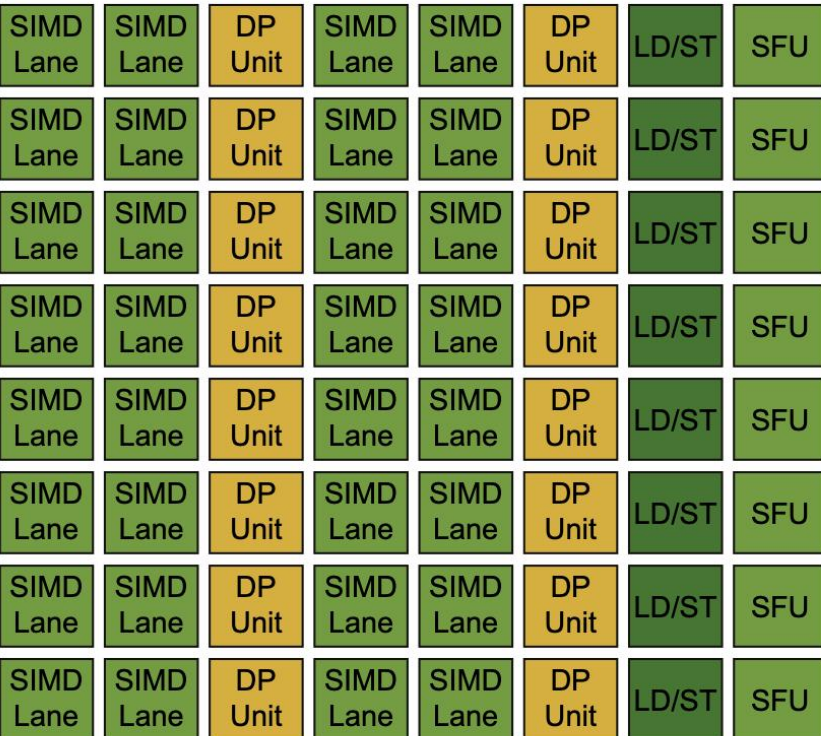
Dispatch
Units

Dispatch
Units

Dispatch
Units

Dispatch
Units

Register File (32,768 × 32-bit)



Instruction Buffer

SIMD Thread Scheduler

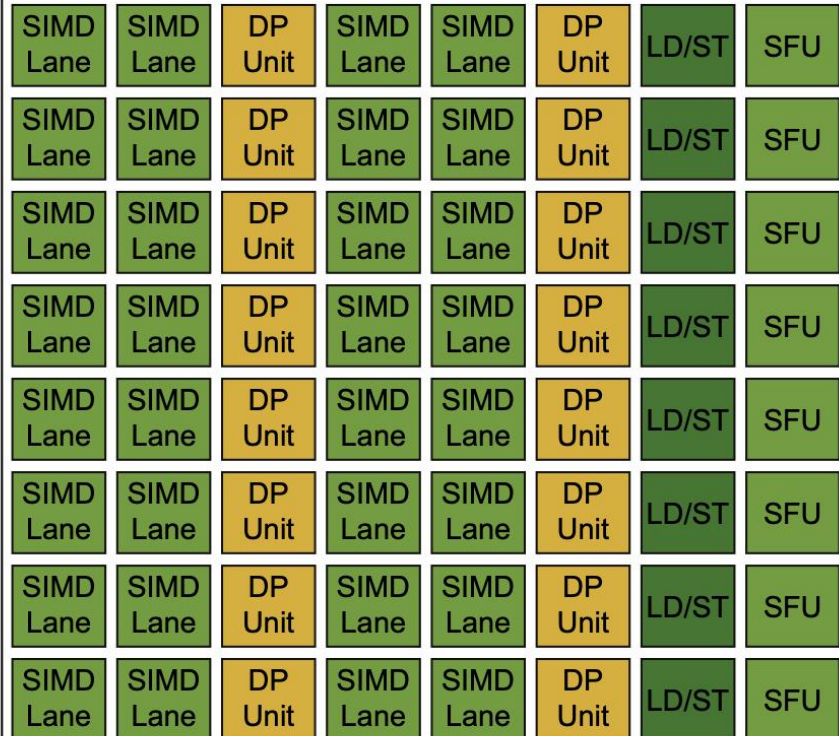
Dispatch
Units

Dispatch
Units

Dispatch
Units

Dispatch
Units

Register File (32,768 × 32-bit)



Pascal GPU

Texture / L1 Cache

Tex

Tex

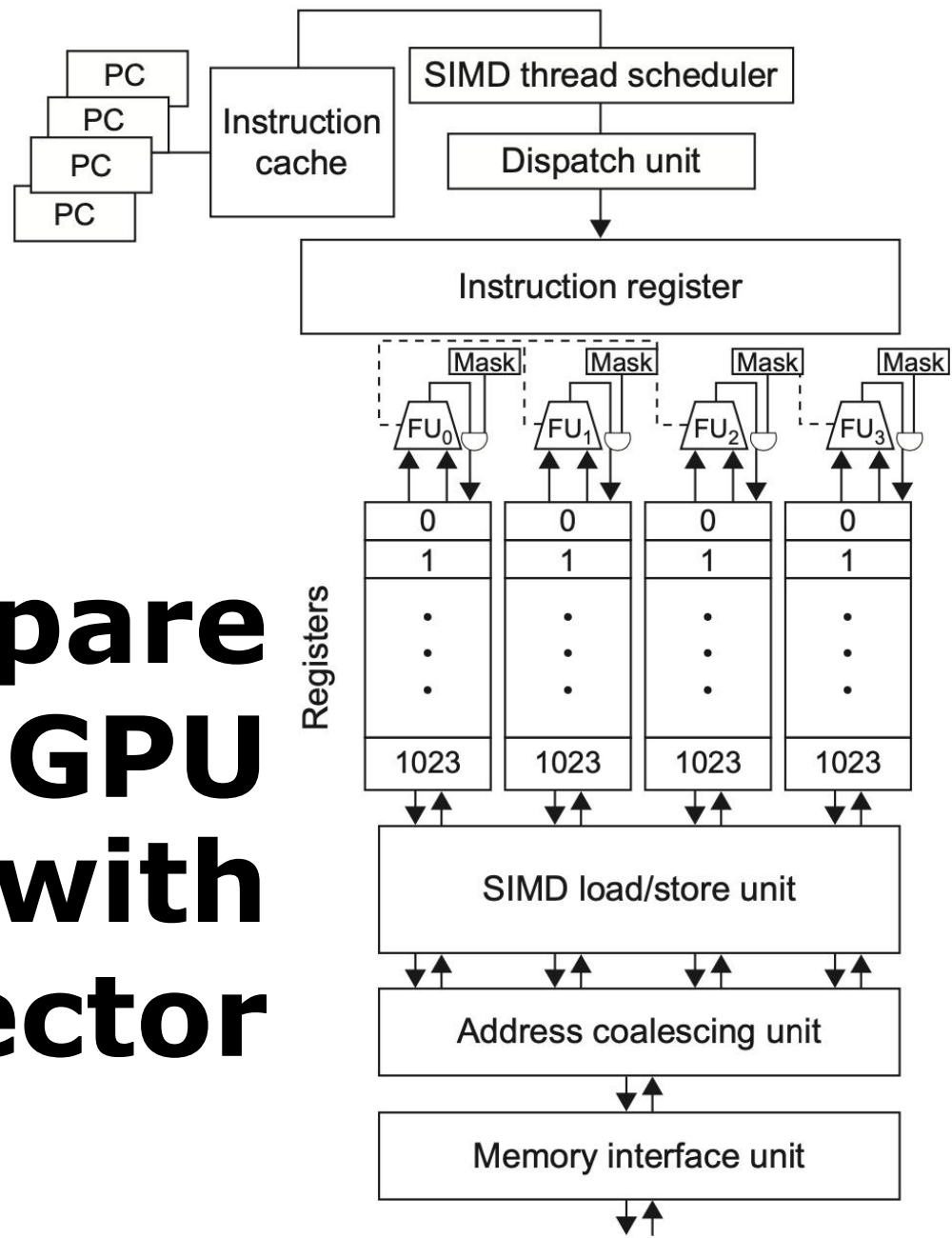
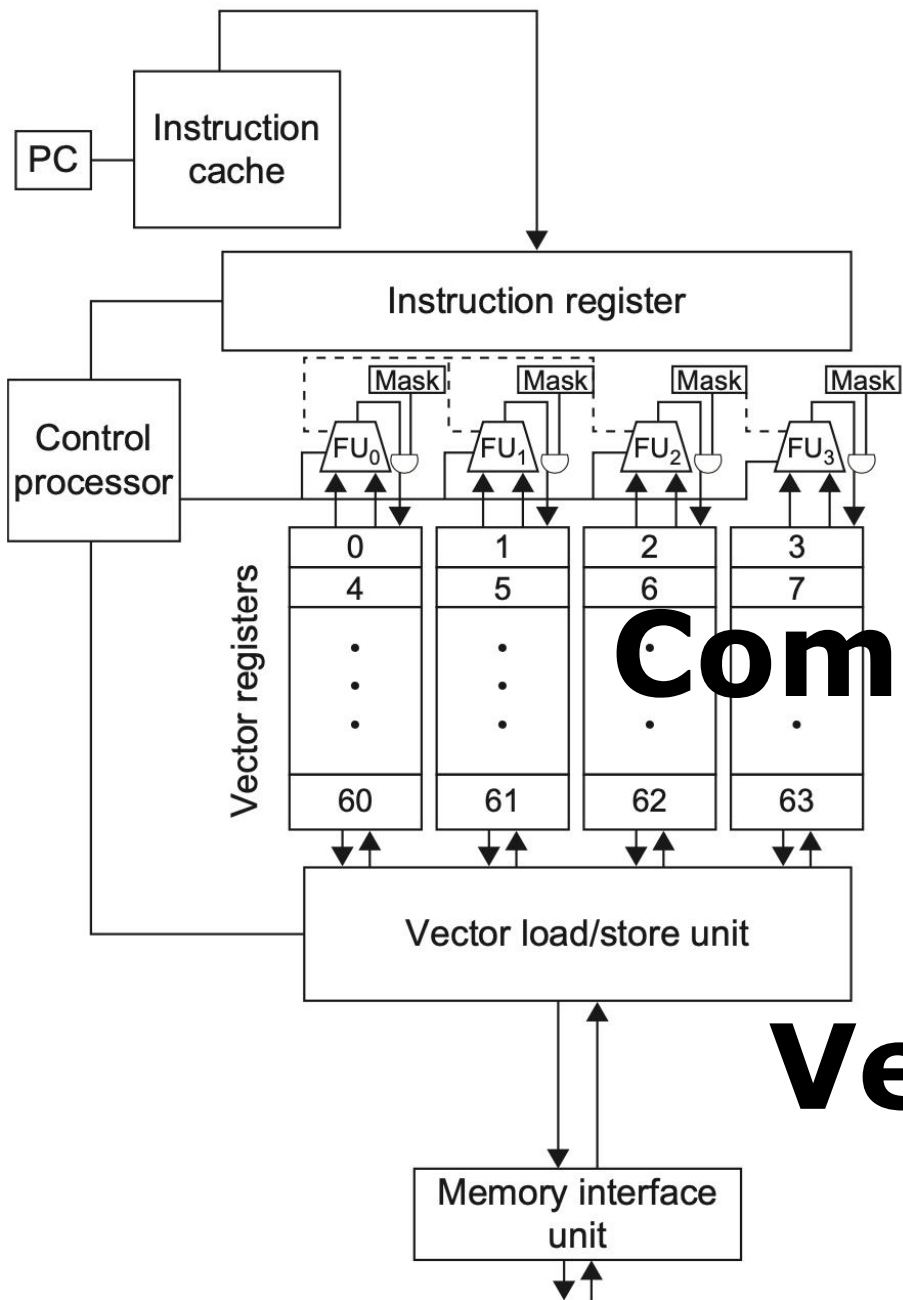
Tex

Tex

64KB Shared Memory

Type	Vector term	Closest CUDA/NVIDIA GPU term	Comment
Program abstractions	Vectorized Loop	Grid	Concepts are similar, with the GPU using the less descriptive term
	Chime	—	Because a vector instruction (PTX instruction) takes just 2 cycle on Pascal to complete, a chime is short in GPUs. Pascal has two execution units that support the most common floating-point instructions that are used alternately, so the effective issue rate is 1 instruction every clock cycle
Machine objects	Vector Instruction	PTX Instruction	A PTX instruction of a SIMD Thread is broadcast to all SIMD Lanes, so it is similar to a vector instruction
	Gather/Scatter	Global load/store (ld.global/st.global)	All GPU loads and stores are gather and scatter, in that each SIMD Lane sends a unique address. It's up to the GPU Coalescing Unit to get unit-stride performance when addresses from the SIMD Lanes allow it
	Mask Registers	Predicate Registers and Internal Mask Registers	Vector mask registers are explicitly part of the architectural state, while GPU mask registers are internal to the hardware. The GPU conditional hardware adds a new feature beyond predicate registers to manage masks dynamically
Processing and memory hardware	Vector Processor	Multithreaded SIMD Processor	These are similar, but SIMD Processors tend to have many lanes, taking a few clock cycles per lane to complete a vector, while vector architectures have few lanes and take many cycles to complete a vector. They are also multithreaded where vectors usually are not
	Control Processor	Thread Block Scheduler	The closest is the Thread Block Scheduler that assigns Thread Blocks to a multithreaded SIMD Processor. But GPUs have no scalar-vector operations and no unit-stride or strided data transfer instructions, which Control Processors often provide in vector architectures
	Scalar Processor	System Processor	Because of the lack of shared memory and the high latency to communicate over a PCI bus (1000s of clock cycles), the system processor in a GPU rarely takes on the same tasks that a scalar processor does in a vector architecture
	Vector Lane	SIMD Lane	Very similar; both are essentially functional units with registers
	Vector Registers	SIMD Lane Registers	The equivalent of a vector register is the same register in all 16 SIMD Lanes of a multithreaded SIMD Processor running a thread of SIMD instructions. The number of registers per SIMD Thread is flexible, but the maximum is 256 in Pascal, so the maximum number of vector registers is 256
	Main Memory	GPU Memory	Memory for GPU versus system memory in vector case

Compare GPU with Vector



**Compare
GPU
with
Vector**

Compare GPU with Multimedia SIMD

Feature	Multicore with SIMD	GPU
SIMD Processors	4–8	8–32
SIMD Lanes/Processor	2–4	up to 64
Multithreading hardware support for SIMD Threads	2–4	up to 64
Typical ratio of single-precision to double-precision performance	2:1	2:1
Largest cache size	40 MB	4 MB
Size of memory address	64-bit	64-bit
Size of main memory	up to 1024 GB	up to 24 GB
Memory protection at level of page	Yes	Yes
Demand paging	Yes	Yes
Integrated scalar processor/SIMD Processor	Yes	No
Cache coherent	Yes	Yes on some systems

Summary

Type	More descriptive name used in this book	Official CUDA/NVIDIA term	Short explanation and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Program abstractions	Vectorizable loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more “Thread Blocks” (or bodies of vectorized loop) that can execute in parallel. OpenCL name is “index range.” AMD name is “NDRange”	A Grid is an array of Thread Blocks that can execute concurrently, sequentially, or a mixture
	Body of Vectorized loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. These SIMD Threads can communicate via local memory. AMD and OpenCL name is “work group”	A Thread Block is an array of CUDA Threads that execute concurrently and can cooperate and communicate via shared memory and barrier synchronization. A Thread Block has a Thread Block ID within its Grid
	Sequence of SIMD Lane operations	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask. AMD and OpenCL call a CUDA Thread a “work item”	A CUDA Thread is a lightweight thread that executes a sequential program and that can cooperate with other CUDA Threads executing in the same Thread Block. A CUDA Thread has a thread ID within its Thread Block
Machine object	A thread of SIMD instructions	Warp	A traditional thread, but it contains just SIMD instructions that are executed on a multithreaded SIMD Processor. Results are stored depending on a per-element mask. AMD name is “wavefront”	A warp is a set of parallel CUDA Threads (e.g., 32) that execute the same instruction together in a multithreaded SIMT/SIMD Processor
	SIMD instruction	PTX instruction	A single SIMD instruction executed across the SIMD Lanes. AMD name is “AMDIL” or “FSAIL” instruction	A PTX instruction specifies an instruction executed by a CUDA Thread

Summary

Type	More descriptive name used in this book	Official CUDA/NVIDIA term	Short explanation and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Processing hardware	Multithreaded SIMD processor	Streaming multiprocessor	Multithreaded SIMD Processor that executes thread of SIMD instructions, independent of other SIMD Processors. Both AMD and OpenCL call it a “compute unit.” However, the CUDA programmer writes program for one lane rather than for a “vector” of multiple SIMD Lanes	A streaming multiprocessor (SM) is a multithreaded SIMT/SIMD Processor that executes warps of CUDA Threads. A SIMT program specifies the execution of one CUDA Thread, rather than a vector of multiple SIMD Lanes
	Thread Block Scheduler	Giga Thread Engine	Assigns multiple bodies of vectorized loop to multithreaded SIMD Processors. AMD name is “Ultra-Threaded Dispatch Engine”	Distributes and schedules Thread Blocks of a grid to streaming multiprocessors as resources become available
	SIMD Thread scheduler	Warp scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution. AMD name is “Work Group Scheduler”	A warp scheduler in a streaming multiprocessor schedules warps for execution when their next instruction is ready to execute
	SIMD Lane	Thread processor	Hardware SIMD Lane that executes the operations in a thread of SIMD instructions on a single element. Results are stored depending on mask. OpenCL calls it a “processing element.” AMD name is also “SIMD Lane”	A thread processor is a datapath and register file portion of a streaming multiprocessor that executes operations for one or more lanes of a warp
Memory hardware	GPU Memory	Global memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU. OpenCL calls it “global memory”	Global memory is accessible by all CUDA Threads in any Thread Block in any grid; implemented as a region of DRAM, and may be cached
	Private memory	Local memory	Portion of DRAM memory private to each SIMD Lane. Both AMD and OpenCL call it “private memory”	Private “thread-local” memory for a CUDA Thread; implemented as a cached region of DRAM
	Local memory	Shared memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors. OpenCL calls it “local memory.” AMD calls it “group memory”	Fast SRAM memory shared by the CUDA Threads composing a Thread Block, and private to that Thread Block. Used for communication among CUDA Threads in a Thread Block at barrier synchronization points
	SIMD Lane registers	Registers	Registers in a single SIMD Lane allocated across body of vectorized loop. AMD also calls them “registers”	Private registers for a CUDA Thread; implemented as multithreaded register file for certain lanes of several warps for each thread processor

Review

- Vector Architecture
- Multimedia SIMD
- GPU